



PDF Download
3737873.pdf
05 January 2026
Total Citations: 18
Total Downloads:
2429

Latest updates: <https://dl.acm.org/doi/10.1145/3737873>

SURVEY

A Survey on Employing Large Language Models for Text-to-SQL Tasks

LIANG SHI, Peking University, Beijing, China

ZHENGJU TANG, Peking University, Beijing, China

NAN ZHANG

XIAOTONG ZHANG, ByteDance Ltd., Beijing, China

ZHI YANG, Peking University, Beijing, China

Open Access Support provided by:

Peking University

ByteDance Ltd.

Published: 10 September 2025

Online AM: 03 June 2025

Accepted: 18 May 2025

Revised: 05 May 2025

Received: 12 November 2024

[Citation in BibTeX format](#)

A Survey on Employing Large Language Models for Text-to-SQL Tasks

LIANG SHI, School of Computer Science, Peking University, Beijing, China

ZHENGJU TANG, School of Computer Science, Peking University, Beijing, China

NAN ZHANG, ZettaData US, Bellevue, United States

XIAOTONG ZHANG, Beijing Bytedance Technology Co Ltd, Beijing, China

ZHI YANG, School of Computer Science, Peking University, Beijing, China

With the development of the Large Language Models (LLMs), a large range of LLM-based Text-to-SQL(Text2SQL) methods have emerged. This survey provides a comprehensive review of LLM-based Text2SQL studies. We first enumerate classic benchmarks and evaluation metrics. For the two mainstream methods, prompt engineering and finetuning, we introduce a comprehensive taxonomy and offer practical insights into each subcategory. We present an overall analysis of the above methods and various models evaluated on well-known datasets and extract some characteristics. Finally, we discuss the challenges and future directions in this field.

CCS Concepts: • **Computing methodologies** → **Natural language processing; Machine translation; Knowledge representation and reasoning; Information extraction;**

Additional Key Words and Phrases: Large language models, Text-to-SQL, prompt engineering, fine-tuning

ACM Reference Format:

Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. 2025. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM Comput. Surv.* 58, 2, Article 54 (September 2025), 37 pages. <https://doi.org/10.1145/3737873>

1 Introduction

In the era of big data, a significant portion of data is stored in relational databases, which serve as the backbone for various organizational data management systems. As the volume of data continues to increase, the capability to efficiently query and leverage this data has emerged as a pivotal factor in enhancing competitiveness across numerous sectors in this era. Relational databases require the use of SQL for querying. However, writing SQL necessitates specialized knowledge, which creates barriers for unprofessional users to query and access databases.

Liang Shi and Zhengju Tang contributed equally to this research.

Authors' Contact Information: Liang Shi, School of Computer Science, Peking University, Beijing, China; e-mail: liang.shi@stu.pku.edu.cn; Zhengju Tang, School of Computer Science, Peking University, Beijing, China; e-mail: zhengju.tang@stu.pku.edu.cn; Nan Zhang, ZettaData US, Bellevue, Washington, United States; e-mail: eric_zhn@hotmail.com; Xiaotong Zhang, Beijing Bytedance Technology Co Ltd, Beijing, China; e-mail: meeme9@163.com; Zhi Yang (corresponding author), School of Computer Science, Peking University, Beijing, China; e-mail: yangzhi@pku.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0360-0300/2025/09-ART54

<https://doi.org/10.1145/3737873>

Text-to-SQL parsing is a well-established task in the field of **natural language processing (NLP)**. Its purpose is to convert natural language queries into SQL queries, bridging the gap between non-expert users and database access. To illustrate, imagine a table named `cities` with three columns: `city_name` (type: string), `population` (type: integer), and `country` (type: string). If we are given the natural language query “Find all the cities with a population greater than 1 million in the United States,” the Text-to-SQL parsing technique should automatically generate the correct SQL query: “SELECT `city_name` FROM `cities` WHERE `population` > 1000000 AND `country` = ‘United States’.” Researchers have made substantial progress in this area. Initially, template-based and rule-based methods [57, 132] were employed. These approaches involved creating SQL templates for various scenarios. While template-based methods showed promise, they required significant manual effort. With the rapid advancement of deep learning, Seq2Seq [101] methods have emerged as the mainstream approach. Seq2Seq [101] models provide an end-to-end solution, directly mapping natural language input to SQL output, which eliminates the need for intermediate steps like semantic parsing or rule-based systems. Among Seq2Seq [101] methods, **pre-trained language models (PLMs)**, which serve as predecessors to **large language models (LLMs)**, show promise in text-to-SQL tasks. Benefiting from the rich linguistic knowledge in large-scale corpora, PLMs became the **state-of-the-art (SOTA)** solution during that period [88].

As model sizes and training data continue to grow, PLMs naturally evolve into LLMs, exhibiting even greater power. Due to the scaling law [49] and their emergent capabilities [114], LLMs have made substantial contributions across diverse domains, including chatbots [20], software engineering [44], and agents [109], and so on. The remarkable capabilities of LLMs have prompted research into their application for text-to-SQL tasks. Current literature on LLM-based text-to-SQL mainly focuses on two main approaches of LLMs: prompt engineering and fine-tuning. Prompt engineering methods take advantage of the instruction-following capability of LLMs to implement well-designed workflows. Moreover, prompt engineering methods frequently utilize **retrieval-augmented generation (RAG)** and few-shot learning to obtain helpful knowledge and demonstrations, and employ reasoning techniques such as **Chain-of-Thought (CoT)** [115] to further enhance performance. Fine-tuning methods follow the “pre-training and fine-tuning” learning paradigm of PLMs [141] and involve training an pretrained LLM on text-to-SQL datasets. There exists a tradeoff between prompt engineering methods and fine-tuning methods. Usually, prompt engineering demands less data but may lead to suboptimal results, while fine-tuning can enhance performance but necessitates a larger training dataset.

This article aims to provide a comprehensive survey of employing LLMs for text-to-SQL tasks and will introduce LLM-based text-to-SQL in the following aspects:

- **Overview:** We provide a brief introduction to LLMs and LLM-based text-to-SQL methods. We also summarize the differences between traditional and LLM-based Text-to-SQL approaches as well as the advantages of LLM-based Text-to-SQL.
- **Benchmark and evaluation metrics:** For benchmarks, we classify them into two categories: benchmarks prior to the rise of LLM and benchmarks in the era of LLMs. We summarize the statistics and characteristics of each benchmark. Additionally, we introduce in detail five benchmarks in the era of LLMs. These datasets were created after the widespread use of LLM and present new challenges such as domain knowledge, large-scale table schemas, diverse perturbations, and real-world enterprise-level scenarios. For evaluation metrics, we introduce metrics frequently used in text-to-SQL tasks. In addition, we introduce some benchmarking studies.
- **Prompt engineering:** We categorize the prompt engineering methods for text-to-SQL into three stages, namely, pre-processing, inference, and post-processing. Pre-processing will

handle the format and layout of questions and table schemas. We also emphatically introduce schema linking techniques in the pre-processing stage. In the inference stage, we explain how LLM-based text-to-SQL methods generate the corresponding SQL query when provided with the user's question and corresponding database schemas. In the final post-processing stage, we introduce how to enhance the performance and stability of LLM-based Text-to-SQL methods after generating SQL queries.

- **Fine-tuning:** We organize this section into three main components, namely, finetuning objectives, training methods, training data and model evaluation. Firstly, we will summarize finetuning objectives in recent LLM-based text-to-SQL articles, which include the SQL generation objective and other objectives aimed at enhancing the performance of different steps. Then frequently used training methods will be introduced. Next, there will be an introduction on how to obtain finetuning data. Finally, the model evaluation methods will be discussed.
- **Model:** We classify LLMs according to whether they are open-source and present the applications of these models in the text-to-SQL domain. Additionally, we analyze the development trend of these LLMs, which reveals the usage frequency of the base models in the LLM-based text-to-SQL methods over time.
- **Analysis:** We conduct a comparative analysis of experimental results from existing studies to further investigate the practical value and applicability of methods and models.
- **Future directions:** We conduct a review of the error analysis of research approaches. Subsequently, we introduce the challenges faced by existing solutions in practical settings and provide some promising future development direction of LLM-based text-to-SQL tasks.

As one of the earliest surveys [43, 68, 139, 147] on LLM-based Text-to-SQL, our work distinguishes itself by introducing a systematic taxonomy of existing methods, providing a comprehensive analysis of the entire LLM-powered Text-to-SQL pipeline, and reinforcing practical utility through concise “Key Takeaways” at the end of each section. We intend this survey to serve as a valuable resource for newcomers to this field and offer valuable insights for researchers.

2 Overview

2.1 LLMs and LLM-based Text-to-SQL

LLMs have emerged as a milestone for NLP and machine learning. The concept of LLMs comes from the practice of continually scaling up the parameter size of PLMs and the volume of training data [141], which results in surprising abilities, known as emergent abilities [114], that are not found in smaller PLMs. One example of emergent capability is few-shot learning [7], meaning that LLMs can complete the downstream tasks with several proper task demonstrations in the prompt without further training. Another example is the instruction following ability [113], with which LLMs have been shown to respond appropriately to instructions describing an unseen task.

Due to the emergent abilities [114] of LLMs and the basic operating principle of LLMs which gradually produce the next word that has the highest probability based on the input prompt [141], prompt engineering becomes one of the two primary streams to apply LLMs to downstream tasks. The representative approaches of prompt engineering are RAG [36], few-shot learning [7], and reasoning [115, 126, 145]. The other stream of LLMs' application to downstream tasks is called fine-tuning, which follows the “pre-training and fine-tuning” learning paradigm of PLMs [141] and aims to boost performance in specific domains and address privacy concerns. The general fine-tuning process mainly involves data preparation, pre-training model selection, model fine-tuning, and model evaluation.

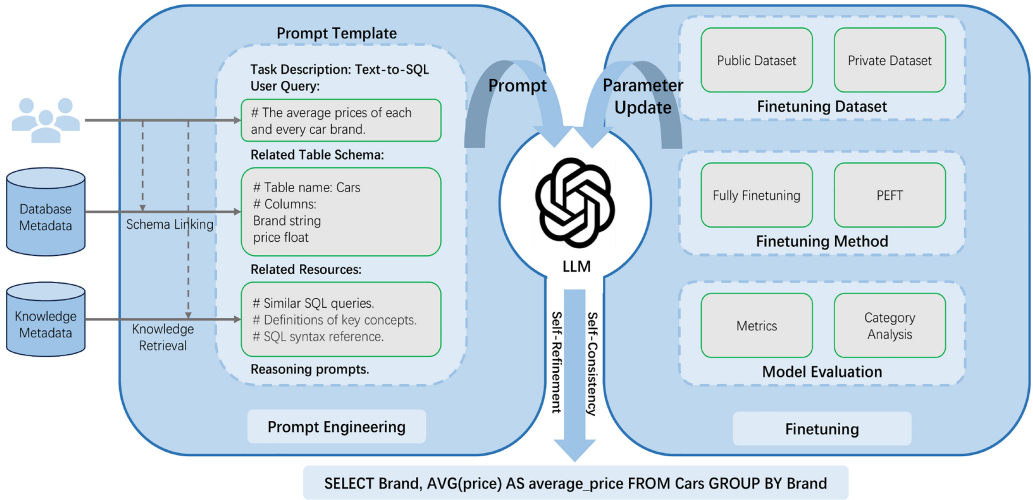


Fig. 1. Framework of employing LLMs in Text-to-SQL.

Text-to-SQL, which is a challenging task in both NLP and database communities and involves mapping natural language questions on the given relational database into SQL queries, has also been revolutionized by the emergence of LLMs. We summarize the general framework of LLM-based Text-to-SQL systems in Figure 1. Based on the two main streams of LLM applications, we categorize the methods used in LLM-based Text-to-SQL into two categories: prompt engineering and fine-tuning. As for prompt engineering methods, we typically design a well-structured prompt encompassing various components, such as task descriptions, table schemas, questions, and additional knowledge, and utilize **in-context learning (ICL)** and reasoning methods at the same time. As for fine-tuning methods, we typically generate or collect text-to-SQL datasets, select appropriate pretrained LLMs and finetuning approaches such as LORA [45], and compare the test results before and after to understand the changes in the model's performance.

2.2 Difference between Traditional and LLM-based Text-to-SQL Approaches

Prior to the widespread of LLMs, there were two primary streams for text-to-SQL methods. One stream involves utilizing the **sequence-to-sequence (Seq2Seq)** model [101], in which an encoder is designed to capture the semantics of the **natural language (NL)** question and the corresponding table schema, and a decoder is employed to generate the SQL query token by token based on the encoded question representation. Some notable methods in this approach include IRNet [47], SQLNet [122], Seq2SQL [144], HydraNet [73], Ryansql [19], Resdsq [58], and ISESL-SQL [63]. The other stream involves finetuning PLMs such as BERT [50], which leverage the extensive knowledge present in large-scale text collections and have proved to be effective in enhancing the performance of downstream text-to-SQL parsing tasks.

We argue that there are two distinct aspects that differentiate traditional text-to-SQL approaches from LLM-based ones:

- **Novel Paradigm:** Traditional text-to-SQL approaches necessitate training, whereas LLMs can often circumvent this requirement. Leveraging the instruction following ability [113] of LLMs, LLMs can accomplish the text-to-SQL tasks with the aid of appropriate instructions and information.

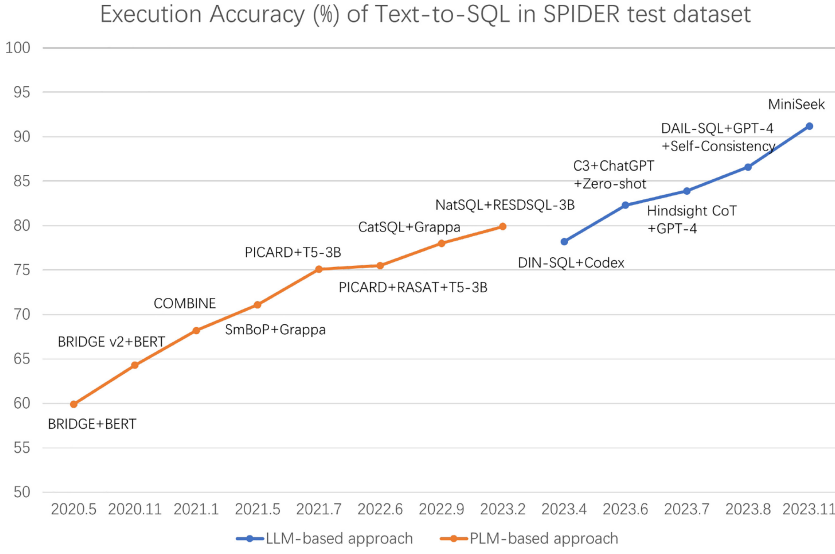


Fig. 2. The evolution of Text-to-SQL approach over time.

- **Uniform Architecture:** According to a previous survey [88], the encoder and decoder in traditional approaches can be designed using diverse architectures such as LSTM [41], Transformer [106], and even GNNs [6]. In contrast, LLMs adhere to a uniform transformer-based architecture, which not only enables easier scaling up but also facilitates a more streamlined implementation.

2.3 Why LLM-based Text-to-SQL

There has been a sharp increase in the number of LLMs employed for text-to-SQL tasks in recent times. After conducting a comprehensive survey of recent articles, we identify several key reasons for this trend, which are summarized below:

- **Enhanced Performance:** Figure 2 illustrates the progression of mainstreaming approaches within the Text-to-SQL domain, as indicated by the Execution Accuracy in the SPIDER test dataset [129]. As we can see, LLM-based methods have significantly improved the SOTA performance, demonstrating the remarkable capabilities of LLM-based methods. The listed references [25, 34, 35, 58, 62, 75, 85, 87, 92, 93] support this observation.
- **Generalization Ability and Adaptability:** As we mentioned above, LLMs have introduced a new paradigm, namely, prompt engineering, which benefits from their ability to follow instructions [113], making LLMs easily transferable to different settings without additional training. Furthermore, the ICL ability [141] of LLMs further enhances their generalization and adaptability capabilities, as they can learn from examples provided to seamlessly fit into various settings.
- **Future Improvements:** LLM-based methods hold a promising future for advancements. Since the global community is prioritizing the enhancement of LLMs, efforts and resources are concentrated towards supporting research on LLMs. This includes scaling up LLMs, creating new prompting methods, generating high-quality and extensive datasets, and fine-tuning LLMs across various tasks. The progress made by the LLM community will undoubtedly and continuously drive LLM-based text-to-SQL methods to new SOTA.

Table 1. Benchmarks Basic Information

Category	Dataset	Proposed time	Statistics	Supplement
single-turn	WikiSQL [144]	2017	80654 hand-annotated examples of questions and SQL queries distributed across 24241 tables from Wikipedia	From Wikipedia
	Spider 1.0 [129]	2018	10,181 questions and 5,693 unique complex SQL queries on 200 databases with multiple tables covering 138 different domains	A large-scale complex and cross-domain semantic parsing and text-to-SQL dataset annotated by 11 Yale students
	CSpider [76]	2019	9691 examples as Spider test set is not publicly available	A Chinese large-scale complex and cross-domain semantic parsing and text-to-SQL dataset translated from Spider
	TABFACT [15]	2019	117,854 manually annotated statements with regard to 16,573 Wikipedia tables	TABFact is the first dataset to evaluate language inference on structured data, which involves mixed reasoning skills in both symbolic and linguistic aspects
	Spider-Realistic [23]	2020	508 examples on 19 databases	Created based on the dev split of the Spider dataset and modified the original questions to remove the explicit mention of column names
	DuSQL [110]	2020	containing 200 databases, 813 tables, and 23,797 question/SQL pairs	A large-scale and pragmatic Chinese dataset for the cross-domain text-to-SQL task
	Fiben [94]	2020	300 NL questions, corresponding to 237 different complex SQL queries on a database with 152 tables	A dataset designed to evaluate the processing of complex business intelligence queries, often involving nesting and aggregation, for NL2SQL systems
	Spider-SYN [83]	2021	Spider-Syn contains 7000 training and 1034 development examples	NL questions in Spider-Syn are modified from Spider, by replacing their schema-related words with manually selected synonyms that reflect real-world question paraphrases.
	KaggleDBQA [53]	2021	272 examples covering 8 DB	A cross-domain evaluation dataset of real Web databases, with domain-specific data types, original formatting, and unrestricted questions
	SPIDER-CG [32]	2022	45,599 examples, around six times the Spider dataset	It split the sentences in the Spider into subsentences, annotate each sub-sentence with corresponding SQL clause and then compose these sub-sentences in different combinations to test the ability of models to generalize compositionally
	ADVETA [83]	2022	depending on which datasets are used	Based on some dataset and add adversarial table perturbation to fool Text-toSQL parsers
	SCIENBENCH MARK [140]	2023	more than 6,000 NL/SQL-pairs	A complex NL-to-SQL benchmark for three real-world, highly domain-specific databases: Research Policy Making, Astrophysics, and Cancer research
	DR-SPIDER [12]	2023	15K perturbed examples	Based on Spider and designs 17 perturbations on databases, natural language questions, and SQL queries to measure robustness from various angles
	BIRD [60]	2023	containing 12,751 text-to-SQL pairs and 95 databases with a total size of 33.4 GB, spanning 37 professional domains	It highlights the new challenges of dirty and noisy database values, external knowledge grounding between NL questions and database values, and SQL efficiency, particularly in the context of massive databases
multi-turn	Spider 2.0 [55]	2024	600 real-world text-to-SQL workflow problems	The databases in Spider 2.0 are sourced from real data applications, often containing over 1,000 columns and stored in cloud or local database systems such as BigQuery, Snowflake, or PostgreSQL. Solving problems in Spider 2.0 frequently requires understanding and searching through database metadata, dialect documentation, and even project-level codebases.
	BIRD-Critic 1.0 [60]	2025	600 tasks for development and 200 held-out OOD tests	It introduces a novel SQL evaluation framework designed to assess LLMs' ability to diagnose and resolve user issues in real-world database environments.
	CoSQL [128]	2019	30k+ turns plus 10k+ annotated SQL queries	A corpus for building cross-domain Conversational text-to-SQL systems
	SPARC [130]	2019	4,298 coherent question sequences	A dataset for cross-domain Semantic Parsing in Context

The “**Supplement**” column contains a brief introduction to each dataset.

3 Benchmark and Evaluation Metrics

High-quality datasets are of crucial importance in the training and testing of Text-to-SQL tasks. In this section, we provide a summary of the mainstream benchmark datasets and evaluation metrics that are utilized in LLM-based text-to-SQL research articles. Additionally, we highlight key benchmarking studies that assess the NL2SQL performance and related capabilities of various LLMs.

3.1 Benchmark

Upon comprehensive examination of articles on LLM-based text-to-SQL, we categorize text-to-SQL datasets into two categories: benchmarks prior to the rise of LLMs and benchmarks in the era of LLMs. We display the details of each dataset in Table 1.

Benchmarks prior to the rise of LLMs. Classic benchmarks in this category, such as WikiSQL [144], Spider 1.0 [129], and KaggleDBQA [53], are widely utilized both before and after the emergence of LLMs. They have made crucial contributions to the advancement of research in the Text-to-SQL domain. As of the time of our writing, Spider 1.0 [129] remains the top choice for evaluating the performance of text-to-SQL methods. Based on these classic benchmarks, a variety of augmented benchmarks have been proposed. For instance, Spider-Realistic [23] creates scenarios where the explicit appearance of the database schema in the questions is eliminated. Spider-SYN [83] replaces schema-related words in questions with manually selected synonyms that mirror real-world question paraphrases. SPIDER-CG [32] splits the sentences in Spider into sub-sentences, annotates each sub-sentence with the corresponding SQL clause, and then composes these sub-sentences in different combinations to test the models’ ability to generalize compositionally. CSpider [76] translates Spider into Chinese to test the performance of Chinese text-to-SQL. ADVETA [83] adds adversarial table perturbation to deceive Text-to-SQL parsers. There are also some other datasets prior to the rise of LLMs, such as Fiben [94] and DuSQL [110] and you can find brief introductions in Table 1.

Benchmarks in the era of LLMs primarily denote benchmarks that emerged after the rise of LLMs. Due to the immense power of LLMs having propelled this field into a new phase, these benchmarks start to focus on more challenging aspects in real-world text-to-SQL tasks, including

domain-specific knowledge, diverse perturbations, large and noisy databases, as well as SQL efficiency. Here, we've summed up five noteworthy benchmark datasets born after the widespread use of LLM.

- *SCIENCEBENCHMARK* [140]. As pointed out in [140], previous datasets such as Spider are not representative of the difficulties encountered when creating an NL interface for a real-world database. Applying a system trained on those datasets to a new domain like astrophysics or cancer research yields poor results, making the adoption of such systems in real-life applications extremely implausible. Hence, SCIENCEBENCHMARK [140] is proposed. It is the first text-to-SQL dataset developed in collaboration with SQL experts and researchers from the fields of research policy making, astrophysics, and cancer research.
- *BIRD* [60]. Most datasets [23, 32, 33, 76, 83, 83, 110, 128–130, 144] focus on the database schema with few rows of database values, leaving the gap between academic research and real-world applications. To address this gap, [60] introduced the BIRD dataset. The BIRD dataset includes 12,751 text-to-SQL pairs and 95 databases, with a total size of 33.4 GB across 37 professional domains. It also highlights new challenges including dirty and noisy database values, external knowledge grounding between NL questions and database values, and SQL efficiency, especially in the context of massive databases. Experimental results indicate that even the most effective text-to-SQL models, i.e., GPT-4 [1], only achieved 54.89% in execution accuracy, which is still far from the human result of 92.96%, proving that challenges still stand.
- *Dr.Spider* [12]. The work in [12] pointed out that Text-to-SQL models are vulnerable to task-specific perturbations, but previous curated robustness test sets usually focus on individual phenomena. To bridge this gap, [12] introduced the Dr.Spider dataset. Based on the Spider dataset [129], Dr.Spider designs 17 perturbations on databases, NL questions, and SQL queries to measure robustness from various angles. To collect more diversified natural question perturbations, [12] leveraged large PLMs to simulate human behaviors in creating natural questions in a few-shot manner. Experimental results indicate that even the most robust model encounters a 14.0% overall performance drop and a 50.7% performance drop on the most challenging perturbation.
- *Spider 2.0* [55]. As indicated in [55], real-world enterprise-level text-to-SQL workflows often involve complex cloud or local data across diverse database systems, multiple SQL queries in various dialects, and a wide range of operations from data transformation to analytics. However, previous datasets lack evaluation in these aspects. Therefore, Spider 2.0 [55] is proposed. It consists of 600 real-world text-to-SQL workflow problems and is sourced from real data applications such as BigQuery, Snowflake, or PostgreSQL.
- *BIRD-Critic 1.0* [60]. Unlike the aforementioned benchmarks that primarily focus on SQL generation, BIRD-Critic 1.0 introduces a novel SQL evaluation framework designed to assess LLMs' ability to diagnose and resolve user issues in real-world database environments. Built upon authentic user-reported problems, this benchmark extends beyond simple SELECT queries to encompass a broader spectrum of SQL operations, thereby better reflecting practical application scenarios. BIRD-Critic 1.0 specifically targets reasoning challenges, as evidenced by the low 38.5% pass rate achieved by even the most advanced models like o1-preview [80].

3.2 Evaluation Metrics

We have summarized the following five metrics:

Exact Set (EM) Match Accuracy is determined by comparing the literal content of the generated SQL and the ground-truth SQL. Specifically, EM compares whether the SQL clauses of the

generated SQL and the ground-truth SQL are consistent. However, there are multiple ways to express the same SQL problem, so the EM metric often underestimates the prediction accuracy of the model.

Execution (EX) Accuracy is concluded by comparing the execution results of the generated SQL and the ground-truth SQL. However, SQL with different logic may yield the same results when executed, so EX may also overestimate the prediction accuracy of the model.

Test-suite (TS) Accuracy [143] creates a small, concentrated database TS from a large number of randomly generated databases. These databases have a high code coverage rate for accurate queries. During the evaluation process, it measures the annotation accuracy of the predicted queries in this TS, effectively calculating the strict upper limit of semantic accuracy.

Valid Efficiency Score (VES) [60] includes SQL execution efficiency in the evaluation scope. The calculation formula is shown in Equation (1), where the hat symbol represents the predicted result, 1 is the indicator function, which is only 1 when the predicted SQL is equivalent to the correct SQL, and R is the square root of the ratio. Intuitively, the higher the correctness rate of the generated SQL, the higher the execution efficiency of the generated SQL, and the higher the VES value.

$$VES = \frac{\sum_{n=1}^N \mathbf{1}(V_n, \hat{V}_n) \cdot R(Y_n, \hat{Y}_n)}{N}, R(Y_n, \hat{Y}_n) = \sqrt{\frac{E(Y_n)}{E(\hat{Y}_n)}}. \quad (1)$$

ESM+ [5] is proposed based on EM Accuracy and applies new rules to LEFT JOIN, RIGHT JOIN, OUTER JOIN, INNER JOIN, JOIN, DISTINCT, LIMIT, IN, foreign keys, schema check, and alias checks compared to EM. [5] compare the performance of nine LLM-based models using TS, EM, and ESM+ and the results indicate that ESM+ offers a substantial improvement by reducing the occurrences of both false positives and false negatives that commonly plague the earlier metrics (EM and TS).

3.3 Benchmarking Studies

Several benchmarking studies have been conducted to evaluate the performance and capabilities of LLMs in the Text-to-SQL task. The work in [89] assesses Codex [14] and GPT-3 [29] models of different sizes using different prompt configurations on Text-to-SQL benchmarks. Their findings demonstrate that generative language models trained on code establish a strong baseline for Text-to-SQL tasks, and that prompt-based few-shot learning can perform competitively with fine-tuning-based approaches. The work in [35] conducts a systematic and extensive comparison of prompt engineering strategies, encompassing aspects such as question representation, example selection, and example organization. Based on their findings, they propose DAIL-SQL [35]. The work in [133] design five distinct tasks, namely, Text-to-SQL, SQL Debugging, SQL Optimization, Schema Linking, and SQL-to-Text, to comprehensively assess LLM capabilities across the entire Text-to-SQL workflow. Another work [125] evaluates schema generation methods, including DIN-SQL [85], C3 [25], RESDSQL [58], and their own schema linking approach. Unlike the aforementioned studies, which primarily focus on prompt engineering, DB-GPT-Hub [146] emphasizes fine-tuning of medium- to large-scale open-source LLMs and provides a modular, extensible codebase that supports a variety of mainstream models and experimental setups.

While these studies provide valuable insights, the rapid evolution of LLM-based Text-to-SQL methods and the frequent updates to models and techniques make benchmarking results quickly outdated. Therefore, we do not delve into specific benchmarking results here and instead refer interested readers to the original works.

4 Prompt Engineering

Prompt engineering, sometimes called ICL, is structuring the instructions that LLM can somehow understand. From the developer's perspective, it means customizing LLM's output on certain tasks by designing prompt words when interacting with LLM. Due to the autoregressive decoding properties [31], most LLMs predict the following text based on all currently visible preceding text which is also called context. We describe the autoregressive decoding as Equation (2), as y_t indicates the next token LLM will output, and $\mathbf{x}$ indicates prompt tokens given by users. The design of prompt words as context will affect the probability distribution of all upcoming tokens, affecting the final generation.

$$y_t = \arg \max P(y_t | y_{1:t-1}, \mathbf{x}). \quad (2)$$

After the comprehensive investigation of related articles, we divide prompt engineering methods on text-to-SQL into the following three stages:

- **Pre-processing.** In practical scenarios, experts engaged in text-to-SQL are often confused by the unclearness of question descriptions and the fuzziness of database schema. This situation also applies to LLMs and urges the pre-processing of the text-to-SQL problems. Our pre-processing part of the prompt engineering method will consist of two parts: first, the abstract representation of question descriptions; second, the selective linking of database schema.
- **Inference.** The so-called inference is defined as generating the corresponding SQL query when presented with the user's question and corresponding database schemas. This stage can be logically divided into two parts: workflow design and demonstration usage. Moving from pre-processing to SQL generation, most work will design their inference workflow either in a custom fashion or based on reasoning patterns, and decide whether to make use of demonstrations. We will start with the introduction of workflow design in text-to-SQL, including the well-known reasoning patterns like Chain-of-Thought [115], Least-to-Most [145], Decomposition, and autonomous agents. After that, we will introduce demonstration methods, including zero-shot methods and few-shot methods. In the few-shot section, we will emphasize the significance of demonstration style and selection.
- **Post-processing.** To enhance the performance and stability of LLM-based Text-to-SQL methods, it is optional to further optimize the generated results after inference. We call these operations post-processing. Common post-processing methods in text-to-SQL include Self-Correction [81] and consistency methods (also known as Self-Consistency [112] and Cross-Consistency [61, 117]).

Figure 3 shows related works classified according to the above three parts, and the pipeline of LLM-based prompt engineering methods for text-to-SQL can be formulated as Equation (3). Detailed stages and insight are as follows:

$$pred_SQL = Post_process(LLM(QuestionRepresentation, Demonstration, Reasoning)). \quad (3)$$

4.1 Pre-processing

At the beginning of text-to-SQL tasks, it's necessary to clearly and comprehensively describe all information needed to solve the problems in the prompt words. The information mainly contains the representation format of questions, database schema, and some task-related knowledge. In the pre-processing part, we will make an introduction to each information part above.

4.1.1 Question Representation. The "Question Representation" in text-to-SQL tasks includes two parts: the NL problem statement and related databases' necessary information. Recent works [3, 89, 107] explore commonly used question representations for questions and databases. The

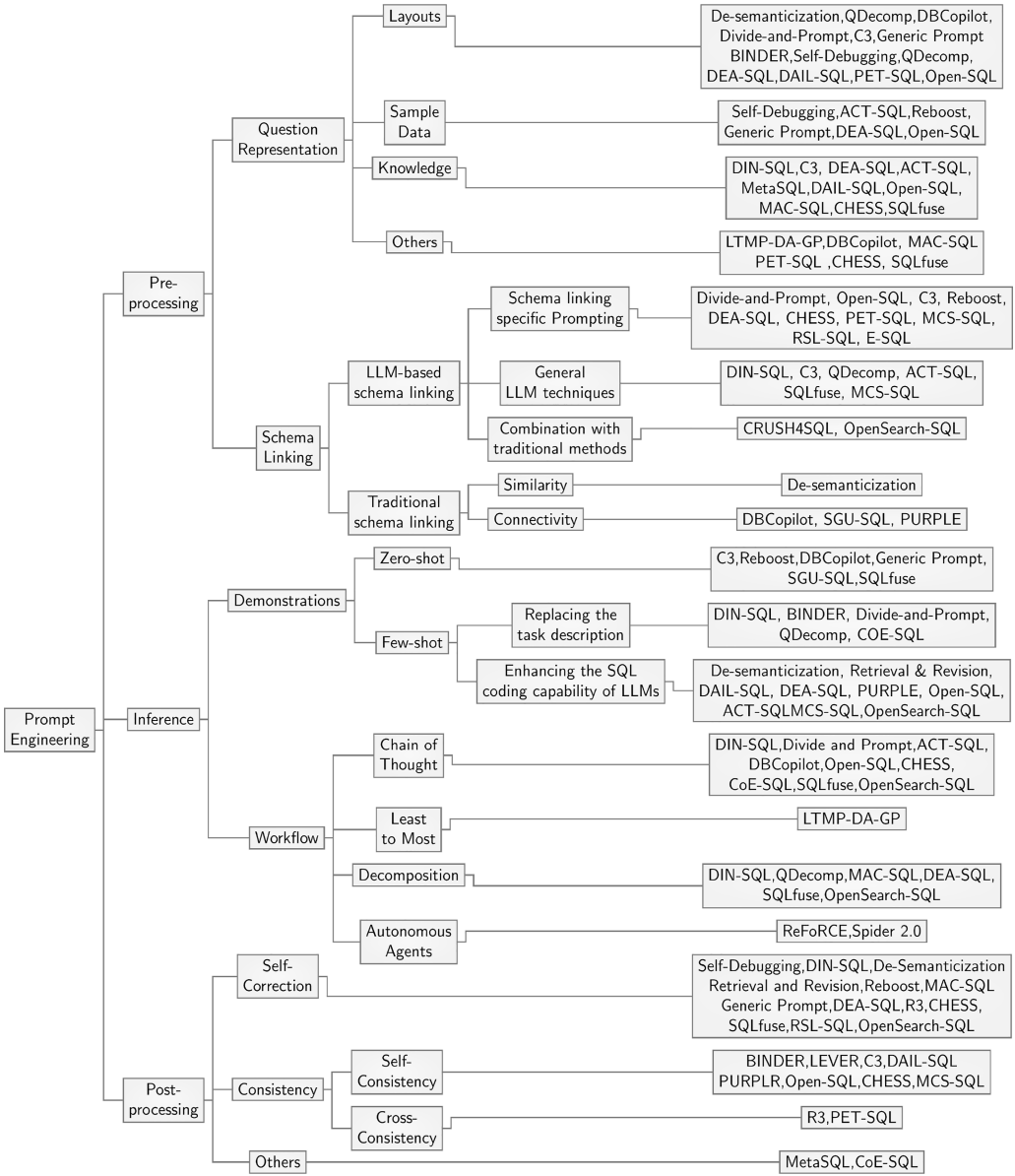


Fig. 3. Taxonomy of prompt engineering.

framework of “Question Representation” is shown in Figure 4. We summarize some typical characteristics of these representations and describe them in the following.

Layouts. One characteristic is the writing style of the question itself and the database structures, which we call the “layout” of the problem. Two common layouts are the OpenAI template layout and the “Create Table” layout.

- *Openai Template layout.* Some work [3, 25, 38, 61, 64, 69, 89, 102, 111] used the Openai prompt template of text-to-SQL task provided in the Codex official API documentation, with each line starting with the comment symbol of SQLite, followed by the form of Table (Column1,

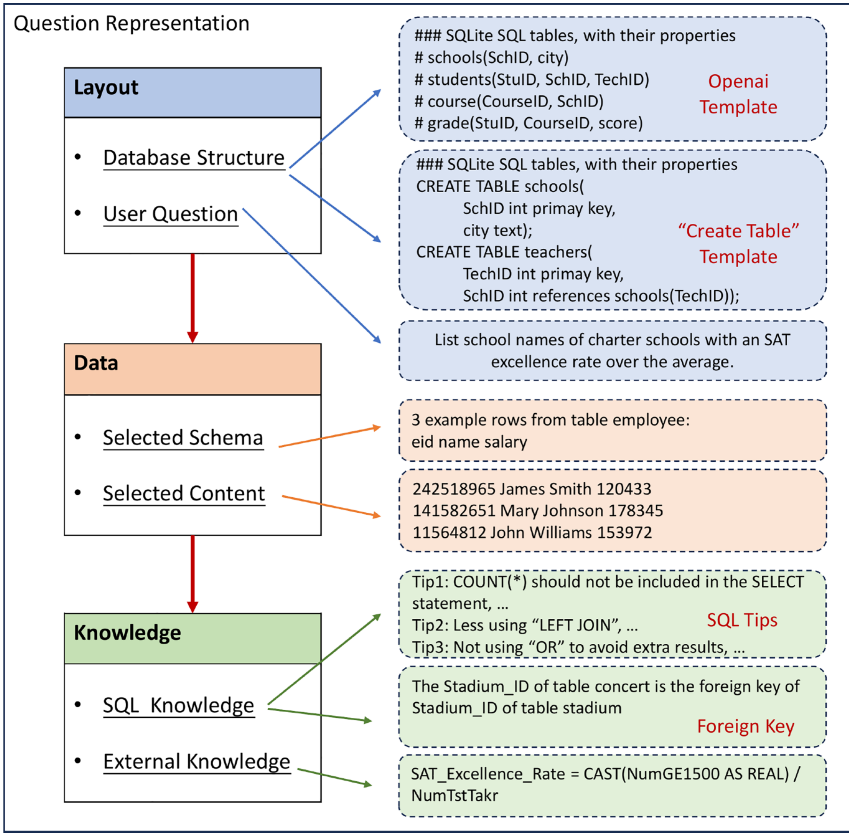


Fig. 4. The framework of "Question Representation". It usually contains three parts sequentially. The *Layout* part includes the question itself and the database structures. The *Data* part includes sampled data from real database content. The *Knowledge* part includes related evidence of some SQL-related priori and other knowledge from the external world. The detail in the figure is just for functional illustration.

Column2,...). After completing the table description, the last row provides the SELECT word. The sample format is shown on the top-right part in Figure 4.

- "Create Table" layout. Other work [16, 17, 35, 60, 77, 89] represented prompt words for database organization as CREATE TABLE creation statements, which is explained on the top-right part of Figure 4. Explicitly including the data type of each column and possible primary and foreign key relationships in the statements will align the data format and perform multi-table connections when LLM generates SQL. Other works [18, 102, 120] used CREATE TABLE statements and partial data to build prompt words, which is more in line with the real situation of data engineers writing SQL.

Sample Data. Apart from the layouts, sample data from real database content is also beneficial for the question representation. The work in [89] represented the prompt as a "SELECT * From Table LIMIT X" statement and the execution result of this SQL statement, where X is an integer and is often taken as 3 based on experience. Some work [3, 16, 17, 99, 120, 135] even directly put sample data into prompt words. The purpose is to enable LLM to understand the sample data in the database and conform to the format of the data itself when generating SQL. [11] modified the form of Sample Data to list column-wise data, to explicitly enumerate categorical value.

Some works combine the layout design and sample data into their prompt to gain the advantage of both. Generic Prompt [3] concatenated the prompt words of Openai Template and Sample Data. LTMP-DA-GP [4] added a random sampling of categorical columns and range information for numerical columns after CREATE TABLE statements. Besides, SQLfuse [138] added one-to-many relationship (a single record in one table is associated with multiple records in another) and enumerating value into the prompt; Some works [39, 52, 85, 100, 107] also use the form of *Table.Column* to list database schema.

Knowledge. Besides layouts and sample data, it's also beneficial to provide related knowledge for LLM with general ability. The knowledge added to prompts can be viewed as a calibration of the current task description, clearing the way for the subsequent SQL generation. We divide knowledge into two categories: one is related to SQL, and the other is related to the problem itself and data.

- *SQL Knowledge.* SQL-related knowledge mainly includes SQL keywords, SQL syntax, and common SQL writing habits. Adding SQL-related knowledge to the prompt for general LLM is like providing junior DBA with an experience handbook, which avoids semantic errors. C3 [25] specifically calibrates the model's bias in SQL style and adds the following instructions in the prompt: using *COUNT(*)* only in specific cases, avoiding using *LEFT JOIN*, *IN* and *OR*, and using *JOIN* and *INTERSECT* instead, and recommending the use of *DISTINCT* and *LIMIT* keywords. DIN-SQL [85] and DEA-SQL [120] notice that some keywords like *JOIN*, *INTERSECT*, and *IN* indicate the hardness of SQL queries, so they design different specifications and hints about the hardness level of the current question based on their judgment. Similarly, Meta-SQL [27] designs three kinds of query metadata to be expressive enough to represent queries' high-level semantics. The types of metadata are operator tag, hardness value, and correctness indicator. To further refine the decision-making capabilities of the critic model, SQLfuse [138] supplements calibration hints that enumerate common errors. These hints serve to preemptively address potential missteps, enhancing the model's ability to generate a superior SQL query. Many works [16, 25, 35, 61, 77, 100, 120, 135] also emphasize primary or foreign key descriptions, attempting to increase LLM's attention to relationships between tables.
- *External Knowledge.* There is also miscellaneous knowledge from the external environment that may be helpful for text-to-SQL tasks, for some jargon or domain-specific words are hard to understand without explanation. MAC-SQL [107] includes some additional informational evidence and database-related requirements before or after the question description. The goal is to clarify schema items and make the LLM clear about the meaning of special terms. Open-SQL [17] utilizes additional descriptive information accompanying each query, as provided by the BIRD dataset [60], that acts as a bridge between human comprehension and the database structure. CHESS [103] uses a context retrieval method to extract descriptions and abbreviations of database catalog, tables, and columns for better performance. SQLfuse [138] specifically designs an "SQL Critic" module to determine the optimal candidate SQL query. The module constructs an external SQL knowledge base from an array of intricate SQL statements and schema in GitHub to better extract knowledge in the external environment.

Following the experiments in [3, 25, 35, 89, 102, 120, 135, 138], we will discuss the observations of performance with different question representations and share some takeaways.

- *Layout.* In the layout perspective, C3 [25] shows a significant drop when changing the layout from clear and structural mode to unstructured mode. When comparing the OpenAI

template and the ‘Create Table’ template, DAIL-SQL [35] claims that the former is better, while QDecomp [102] argues that they are equally matched.

- *Sample Data.* The work in [3, 89] provides performance statistics with each layout, indicating that sample data is pluggable and beneficial to other layouts. ACT-SQL [135] states that more samples are not always better.
- *Primary and foreign keys.* Many works show the importance of primary and foreign keys in the question representation. The ablation study in [61, 120, 135] claims that removing foreign keys will significantly decrease performance. SQLfuse [138] also puts primary keys as the second contributor to the accuracy gain. In DAIL-SQL [35], some LLMs exhibit gain while others see a drop after adding foreign keys.
- *Knowledge.* SQLfuse [138] and DEA-SQL [120] show in their ablation study that the performance degrades significantly without extra information on SQL knowledge and external commonsense.

Based on the discussion above, we are ready to give some takeaways on the choice of question representation methods.

Key takeaways

- Structural layouts (like the Openai template and the ‘Create Table’ template) are better than the unstructured layouts, while different structural layouts are equally matched.
- Sample data is effective, pluggable, and worth consideration when the context length is sufficient.
- Primary and foreign keys have significant effects and are critical and applicable in complex scenarios.
- The knowledge part is important for LLM to understand some SQL evidence and hidden concepts in the question and align with commonsense.

4.1.2 Schema Linking. There are often situations where a problem involves multiple columns in multiple tables, but each table only uses specific columns. Redundant and unrelated schema items probably distract LLM from identifying the right items, calling for schema linking. Schema linking is the subtask within the text-to-SQL process to specify the tables and columns in the database that correspond to the phrases in the given query. Tables and columns can be specified together or separately [74]. Schema linking is crucial in LLM-based text-to-SQL pipelines for two reasons. First, it brings about shorter token lengths. On one hand, for a large database, it is impractical to prompt all the table descriptions into the LLM. On the other hand, by reducing the length of the table schema, we can potentially enhance the attention and effectiveness of the LLMs. Second, research has shown that a significant number of failures in LLM-based text-to-SQL stem from the inability to correctly identify column names, table names, or entities mentioned in questions. By utilizing schema linking techniques, we can improve the performance and even facilitate cross-domain generalization and synthesis of complex queries.

After conducting a comprehensive investigation of schema linking techniques employed in recent LLM-based text-to-SQL articles, we have categorized them into two distinct categories, namely, LLM-based schema linking methods and traditional schema linking methods. Figure 5 shows an overview of our surveyed schema linking methods.

LLM-based schema linking methods refers to methods that utilize LLMs to perform the schema linking task. There are three main approaches:

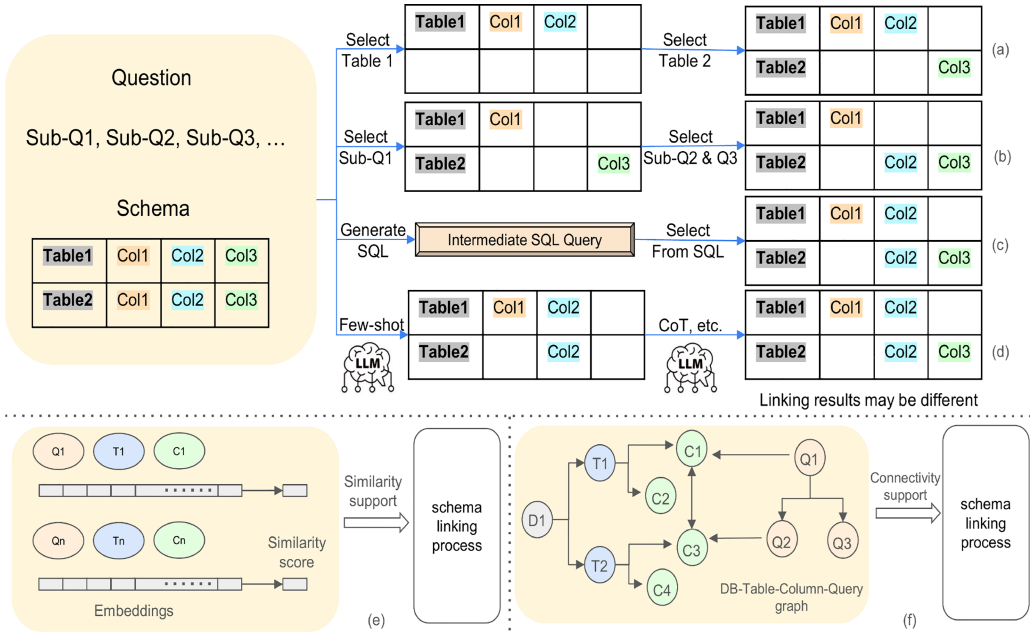


Fig. 5. An overview of schema linking methods used in LLM-based text-to-SQL articles. (a), (b), (c), and (d) are LLM-based schema linking methods, while (e) and (f) are traditional schema linking methods. (a) and (b) correspond to prompting LLMs in specific steps designed for schema linking. (c) corresponds to using SQL to guide schema linking. (d) corresponds to enhancing LLM-based schema linking performance by utilizing general LLM techniques. (e) and (f) are similarity methods and connectivity methods, respectively.

- *Prompting LLMs in specific steps designed for schema linking.* The simplest way is to directly prompt LLMs to do schema linking, as Divide-and-Prompt [69] and Open-SQL [17] do. To further improve the performance in this way, some studies [25, 61, 99, 103, 120] designed more complex paths which prompt LLMs in specific steps. C3 [25] and MCS-SQL [54] break down schema linking into two steps. They first instruct the LLM to recall tables and then retrieve the columns within the candidate tables. Reboost [99] adopts the same approach as C3 [25] but enriches table and column selection by including query descriptions and column explanations. DEA-SQL [120] explored first identifying elements in a query and then utilizing them to filter the schema. CHESS [103], on the other hand, follows a three-step paradigm of “column filtering, table selection, and final column filtering”. PET-SQL [61] emphasizes that LLMs excel in writing SQL compared to the schema linking sub-task, proposing an initial step of composing corresponding SQL queries and then extracting tables and columns from these queries. RSL-SQL [10] introduces a bidirectional schema linking approach to consider both full schema and SQL generated by the LLM, combining the methods and advantages of MCS-SQL [54] and PET-SQL [61]. Also equipped with the guidance of the pre-generated SQL candidate, E-SQL [8] chooses to enrich the NL query by directly incorporating relevant database items and conditions into the question and SQL generation plan.
- *Enhancing LLM-based schema linking performance by utilizing general LLM techniques.* As LLM-based schema linking methods are driven by LLMs, several studies [25, 85, 102, 135, 138] explored enhancing schema linking performance by leveraging general LLM techniques, such as few-shot learning [7], CoT reasoning [115], self-consistency voting [112], and

fine-tuning. DIN-SQL [85] randomly selects some examples to guide schema linking and utilizes “Let’s think step by step” to further improve performance. C3 [25] employs self-consistency [112] to enhance performance stability. MCS-SQL [54] makes different prompts to do schema linking and uses a union operation to include the selected schema items. QDe-comp [102] and ACT-SQL [135] also use few-shot learning to guide schema linking but place more emphasis on example construction. Concretely, given a sub-question and its corresponding SQL, they annotate all table-column pairs mentioned in the SQL as ground truth. In addition, ACT-SQL [135] also employs an embedding model to identify relationships between phrases and schemas, which are then used to construct examples in a CoT style. Besides the above prompting methods, SQLfuse [138] employs finetuning to enhance the LLM’s schema linking performance.

- *Integrating LLMs into traditional schema linking methods.* Besides the above methods, there is another option available, which involves integrating LLMs with traditional schema linking approaches. One such example is CRUSH4SQL [52], which takes advantage of the hallucination capability of LLMs. It first generates a DB schema based on the given query through hallucination, and then employs similarity-based retrieval to select related schemas using the generated schema as a reference. OpenSearch-SQL [119] utilizes vector retrieval after employing an LLM to select schema items to control the similarity between the original query and the selected items.

Traditional schema linking methods refer to schema linking techniques without relying on LLMs. Our thorough analysis reveals that LLM-based text-to-SQL research articles utilize two main streams of these traditional schema linking methods:

- *Similarity-based Retrieval Methods.* To retrieve relevant schemas, the basic idea is to determine the similarity between the query and the schema information and retrieve the most similar schemas. A common implementation employs PLMs such as BERT [50] and RoBERTa [71] to encode both queries and schema elements into embedding vectors for similarity comparison. One such example is De-semanticization [39], which calculates similarity by identifying direct matches between each token in the question and each item in the schema, as well as identifying correspondences between question words and specific database values.
- *Connectivity methods.* Besides similarity, connectivity is another essential consideration, as the tables and columns used should exhibit a relationship or connection between them. Some recent works, like DBCopilot [111], PURPLE [90], and SGU-SQL [137], have explored this concept by utilizing graph-based approaches. DBCopilot [111] first constructs a graph to represent the underlying schema structure of all databases and their tables, and then trains a Seq2Seq model as a router routing over massive databases to get a set of schemas. PURPLE [90] incorporates graph creation based on foreign-primary key connections to enhance the interconnectivity of the retrieved relevant schemas. SGU-SQL [137] constructs a query-schema graph by combining the query structure and database structure based on the topic concepts in the query, predefined relations in the query, database schema as well as the table/column names present within the schema. Then it trains a model to bind the query nodes with corresponding schema nodes.

With the development of LLMs and the enlargement of the context window, more and more schema items can be put into the prompt of text-to-SQL tasks. The experiments in Distillery [74] show that as the model’s SQL generation capability improves, its sensitivity to the presence of irrelevant columns as contextual information for generation decreases. The importance of schema linking diminishes as costs decrease, context windows widen, and generation capabilities improve.

However, schema linking is still useful in most practical cases because the entire schema in real-world data-warehousing scenarios often exceeds the context window.

Key takeaways

- Most works prefer CoT or decomposition reasoning as the foundational workflow.
- The mainstream LLM-based schema linking methods comprise prompting LLMs in specific steps and enhancing performance by utilizing general LLM techniques (such as few-shot learning, CoT reasoning, self-consistency voting, and finetuning).
- Both similarity and connectivity are worthy considerations.

4.2 Inference

Given the question and schema in a certain form, the next step is to generate potential answers to the problem. Considering the complexity and high requirement for accuracy of text-to-SQL tasks, it's difficult to get satisfactory answers by letting LLM directly generate SQL responses to the problem. We investigate related works comprehensively and find two techniques that can help generate the right and high-quality SQLs: workflow and demonstrations.

4.2.1 Workflow. The simplest workflow is to generate SQL directly from the constructed question and schema by only one interaction with LLMs, which somehow overestimates the ability of general LLMs in professional fields. Just like people tend to break down a complex task into several simple sub-tasks or steps, prompt-engineering-based methodologies usually design specific inference workflows for generating responses to queries using LLMs. Workflows in text-to-SQL tasks can be categorized based on distinct reasoning patterns. The following paragraphs will introduce typical workflows in our scenario and compare their strengths in different contexts.

CoT. The most famous reasoning style, CoT [115], involves a series of intermediate reasoning steps and typically begins with the phrase “Let’s think step by step” to elicit chain thinking. The reasoning steps are like clause by clause and keyword by keyword, and outputs the steps in a single pass. DIN-SQL [85] employs human-designed steps in CoT for its complex class of questions. For nested complex questions, the CoT steps usually encompass several sub-questions of the original question, because they match the corresponding keywords or clauses in the SQL. Divide-and-Prompt [69] generates SQL in a clause-by-clause manner, a variant of the CoT pattern. CoE-SQL [136] also proposes a variant of CoT, Chain-of-Edit, which describes 14 edit rules of SQL statements: editing “SELECT” items, editing “WHERE” logical operator, and so on. Furthermore, ACT-SQL [135] proposes Auto-CoT to generate CoT exemplars automatically, addressing the high cost of manually labeling CoT prompts. Open-SQL [17] designs the CoT template that employs a skeleton-based query framework as the intermediate representation.

Least-to-Most. The Least-to-Most [145] reasoning style is another widely utilized approach in the design of text-to-SQL workflows. Different from CoT that usually focuses on the SQL part, Least-to-Most follows the principle that sub-problems can be reduced to the original problem syntactically and semantically. LTMP-DA-GP [4] exemplifies this category. The article adopts Least-to-Most to decompose the NL query, map NatSQL [34] to the decomposition, and generate SQL from NatSQL.

Decomposition. Apart from the above reasoning methods, a straightforward yet effective workflow exists that primarily decomposes the generation task into customized interactions with LLM, employing various techniques to address challenges at each stage [51]. QDecomp [102] first claims this category by proposing the decomposition prompt method. The decomposition of the SQL generation task can be parallel or sequential. Starting from the error analysis of SQL

statements, DIN-SQL [85] classifies SQLs into three levels of complexity: *Easy*, *Nested Complex*, and *Non-Nested Complex*. This is the parallel decomposition for the classification of the task. MAC-SQL [107] introduces a framework comprising a selector, a decomposer, and a refiner. The decomposer agent generates a series of intermediate steps (i.e., sub-questions and SQLs) before predicting the final SQL. This is the sequential decomposition. DEA-SQL [120] follows the global decomposition steps of Information determination, Classification and Hint, SQL generation, Self-correction, and Active learning. OpenSearch-SQL [119] also involves five modules: Preprocessing, Extraction, Generation, Refinement, and Alignment. SQLfuse [138] organizes its SQL generation prompt in the decomposition pattern to incorporate prior schema information and iteratively execute SQL checking and error correction within a single prompt. R3 [117] proposes a consensus-based decomposition system for text-to-SQL tasks. The system consists of an SQL writer and several reviewers of different roles. After several rounds of “negotiation” between the SQL writer and reviewers, there will be a consensus, and the final answer will be decided.

Autonomous Agents. Inspired by ReAct framework [127], autonomous agents can be adapted to the Text-to-SQL domain. Different from “multi-agent” frameworks claimed by much work, the autonomous-agent workflow treats the LLM as the brain and lets the LLM control the interaction with the environment, humans, and other LLM-based agents. An autonomous-agent framework usually features multi-turn interactions, extended inference time, long-term memory, and conditional termination. Spider 2.0 [55] proposes Spider-Agent, which is primarily focused on database-related coding tasks and projects. Spider-Agent allows for multi-turn interactions with the database via command-line interfaces, and automatically terminates if it outputs the same result three times in a row or a timeout is triggered. REFORCE [22] also proposes an agent-based framework with a self-refinement workflow and more domain-specific techniques like table compression, format restriction, and task description. The autonomous agents will not terminate until self-consistency, maximum iterations, or empty results.

CoT, Least-to-Most, Decomposition, and autonomous agents are all useful in the performance of text-to-SQL tasks. At the same time, they have their advantages in different contexts [51]. CoT generates SQL queries in a step-by-step manner, like clause by clause and keyword by keyword, and outputs the steps in a single pass. Least-to-Most mainly derives the sub-questions for problem reduction. Both CoT and Least-to-Most imitate humans’ thinking processes and output understandable answers. Decomposition makes use of various techniques to solve the whole task, and it is suitable when users need additional preparations before constructing SQL queries or post-processing after the SQL query is generated. The preparations include schema linking, example selection, difficulty level judgment [85], SQL refinement [107, 120], and any other customized operations. Autonomous agents are an emerging method and show great potential in real-world tasks [55] by letting LLMs think and interact with each other multiple times with long-term memory.

Key takeaways

- Most works prefer CoT or decomposition reasoning as the foundational workflow.
- It’s recommended to design customized variants of CoT compared with the original CoT.
- Decomposition can benefit SQL generation in a sequential or parallel way.
- Alternative patterns are open for exploration.

4.2.2 Demonstrations. In the workflow of LLM-based text-to-SQL methods, demonstrations are frequently employed to enhance the performance of SQL generation by incorporating several demonstrations. Depending on whether demonstrations are appended, these methods can be

classified into two categories: zero-shot methods and few-shot methods. Examples for zero-shot LLM-based text-to-SQL methods include C3 [25], ReBoost [99], DBCopilot [111], Generic [3], SGU-SQL [137], and SQLfuse [138]. Although zero-shot methods have the advantage of saving the tokens of LLMs, text-to-SQL is a complex task that may be relatively less familiar from the model's perspective. Merely modifying the instructions cannot effectively solve this complex task. In contrast to zero-shot, few-shot has the ability to learn the pattern of tasks from examples and does not solely depend on the pre-trained knowledge of the model. This characteristic enhances the performance and adaptability of LLMs. After conducting a comprehensive investigation of demonstrations employed in recent LLM-based text-to-SQL articles, we have categorized the functions of demonstrations into two distinct categories, including:

- *Replacing the task description.* As mentioned earlier, to handle complex text-to-SQL tasks, studies explore diverse workflows that comprise specific steps or involve newly defined sub-tasks that did not previously exist. For example, when generating SQLs for non-nested complex questions [85], DIN-SQL [85] follows a workflow in the style of CoT with fine-designed steps. It stipulates that first, the NatSQL [34] corresponding to the problem is generated as the intermediate representation. Subsequently, the final result is generated based on this NatSQL [34]. Another example is BINDER [18], which augments programming languages (such as SQL) with language model API call functions and employs an LLM to translate NL queries into extended SQL language. However, both DIN-SQL's [85] fine-designed steps and BINDER's [18] translation are unfamiliar from the model's perspective and are not easily described by human language prompts. In this situation, both of them make use of the ICL capability of LLMs and provide several examples. This significantly simplifies the instruction design in prompts. This idea is also employed by Divide-and-Prompt [69], QDecomp [102], and COE-SQL [136].
- *Enhancing the SQL coding capability of LLMs.* Appropriate demonstrations can greatly enhance the performance of LLMs [7] and experiments have found that LLMs are highly sensitive to sample selection, and choosing inappropriate samples may even produce negative effects. To maximize performance, studies [17, 35, 37, 39, 54, 90, 120, 135] explore how to select appropriate demonstrations. The simplest way is to retrieve examples whose questions have similar semantic meanings. However, questions with different database schemes can vary significantly, even if their underlying intentions are similar and the corresponding SQL queries show similarities [39]. To bridge this gap, studies such as De-semanticization [39], Retrieval and Revision [37], DAIL-SQL [35], DEA-SQL [120], and PURPLE [90] first mask domain-specific words in the original question to obtain the skeletons of the query. Then, they retrieve examples whose question skeletons have similar semantic meanings. Among them, PURPLE [90] further designs four levels of SQL skeleton abstraction and focuses on more coarse-grained retrieval. Some studies [17, 35, 37, 54] try to use more information for retrieving examples. Retrieval and Revision [37] simplifies NL questions by prompting the LLM with instructions and retrieves examples utilizing both the original question and the simplified question, which avoids the frustration of unusual questioning styles and enhances the syntax and wording variety in the repository. DAIL-SQL [35] retrieves examples utilizing both NL questions and corresponding SQL queries, while Open-SQL [17] retrieves examples utilizing NL questions, database schema, and corresponding SQL queries. OpenSearch-SQL [119] proposes the mask question similarity to select similar queries. MCS-SQL [54] uses two distinct selection strategies to leverage both question similarity and masked question similarity. Besides retrieving similar examples, diverse examples may also be helpful. One example is ACT-SQL [135]. It utilizes randomly selected examples as well as examples similar

to the current question. In addition to example selection, token-cost is another consideration. DAIL-SQL [35] considers the tradeoff between accuracy and token-cost and provides three example styles, namely, the combination of the query, schemas and corresponding SQL, the combination of the query and corresponding SQL, and only SQL. DAIL-SQL [35] chooses the combination of the query and corresponding SQL to reduce the token length of examples.

Key takeaways

- One straightforward way of task description in a prompt is to utilize demonstrations. The more complex the workflow, the greater this method's advantage.
- Question skeletons can capture the question intention more effectively than the original questions.
- A tradeoff between accuracy and token-cost needs to be taken into account.

4.3 Post-processing

To further improve the performance and stability of LLM-based Text-to-SQL methods, studies apply post-processing to the generated SQLs. After a comprehensive investigation, we summarize these methods into two categories: self-correction and consistency.

4.3.1 Self-Correction. After LLM generates an answer, the Self-Correction method uses rules under specific questions and tasks to have LLM check the correctness of the answer; In the text-to-SQL scenario, the Self-Correction method can use SQL-related rules for checking, and can also provide LLM with the results or error logs generated by running SQL statements for inspection. [3] noticed the subtle difference in extra spaces in table values and let LLM recheck them. RSL-SQL [10] and OpenSearch-SQL [119] repeat the SQL generation until the results are not empty or the maximum dialogue rounds are reached. The approach of [37–39] is to change the small number of schemas in the original prompt words to a full number of schemas and generate SQL again if the SQL generated by LLM cannot run. In [37, 85, 99, 103, 107], the incorrect SQL and the information indicating the execution error will be used as prompts for LLM to regenerate the SQL. [120] analyzed several significant error points of field matching and SQL syntax, then designed specific prompts to correct them. [16] designed unit tests, code explanation, and execution results for the model to refine its response. [138] proposed the SQL Critic module and adopted a few-shot ICL strategy that leverages examples from an external SQL knowledge base, enriched with hindsight feedback.

4.3.2 Consistency Method. The Self-Consistency method [112] mainly adopts a majority voting strategy, allowing the same LLM to generate multiple answers to the same question by setting a certain temperature. Then LLM selects the answer that appears the most frequently as the final answer. In some work [17, 18, 25, 35, 100, 103, 119], the Self-Consistency method was directly used to perform a major vote on the SQL generated by its LLM, achieving good performance improvement. [78] proposed to rerank multiple answers and trained a verifier to check the generated code. PURPLE [90] utilized the majority vote based on execution results of generated SQLs. MCS-SQL [54] utilizes multiple prompts to generate a certain number of SQLs for each prompt, and then selects the most accurate query among the candidates based on the confidence scores and execution time.

Unlike Self-Consistency, the Cross-Consistency method uses several different LLMs or agents to generate answers respectively or check the validity of SQLs. PET-SQL [61] proposed this method to instruct several LLMs under lower temperatures to generate SQLs and then make votes across

the SQLs' executed results. To leverage the advantages of respective strengths, [103] used Self-Correction and Self-Consistency sequentially in its post-processing stage. R3 [117] adopted a multi-agent framework to provide suggestions on SQL from agents with different expertise in a loop way, like the combination of Cross-Consistency and Self-Correction.

4.3.3 Others. DEA-SQL [120] discovers that the model is more prone to errors for certain problem types (e.g., extremum problems), and then utilizes active learning [96] to represent the three fixed error cases and to identify whether the generated SQL should be modified, which better meets the requirements. OpenSearch-SQL [119] focuses on the consistency alignment in the process of SQL generation and adds an alignment module to check each module in the framework aligns with specific requirements.

Key takeaways

- In the text-to-SQL tasks, most works equipped with Self-Correction focus on the 'refine' parts like some handmade schema-modify rules and execution logs, while the 'critic' parts like code explanation and answer judgment have more room for exploration.
- Self-Consistency has the advantages of adaptability, convenience, and good performance. However, it needs more interactions with LLM and costs more.
- Cross-Consistency lets multiple LLMs participate, reducing the prejudice drawback of a single LLM.
- It's promising to use both Self-Correction and consistency methods in the post-processing stage.

5 Fine-tuning

LLMs have demonstrated remarkable performance on Text-to-SQL tasks through prompting methods such as RAG, ICL, and CoT [115]. Nevertheless, these methods heavily rely on powerful yet closed-source LLMs like GPT-4 [1], which may cause privacy concerns [59, 123] since data needs to be transmitted to closed-source model providers. Therefore, we should also pay attention to the application of open-source LLMs in Text-to-SQL tasks. Compared to large closed-source LLMs like GPT-4 [1], open-source LLMs suffer from limited reasoning and instruction-following capabilities. This is due to several factors, including smaller model sizes, a pre-training corpus that is both smaller in quantity and less qualified in quality, as well as less extensive pretraining efforts. Additionally, SQL-related content typically accounts for only a minuscule fraction of their entire pre-training corpus [59], potentially resulting in a subpar performance in text-to-SQL tasks. Therefore, the mainstream of application of open-source LLMs in Text-to-SQL tasks is fine-tuning.

In the context of LLM-based text-to-SQL, fine-tuning involves taking a base LLM, for instance, code-llama [91], and conducting further training on text-to-SQL related tasks, typically the SQL generation task, by utilizing a text-to-SQL dataset in a specific scenario. Fine-tuning in the context of LLM-based text-to-SQL can be formulated as Equation (4). Given a base model M and a fine-tuning dataset $\tau = \{(q_i, d_i, gt_i)\}$, where q_i represents the user query, d_i is the corresponding database information, and gt_i is the ground truth SQL query. The objective is to minimize the difference, as measured by the loss function $Loss$, between the predicted SQL query $M(q_i, d_i)$ and the ground truth SQL query gt_i .

$$\min_M = \sum_{i=0}^{|\tau|} Loss(M(q_i, d_i), gt_i). \quad (4)$$

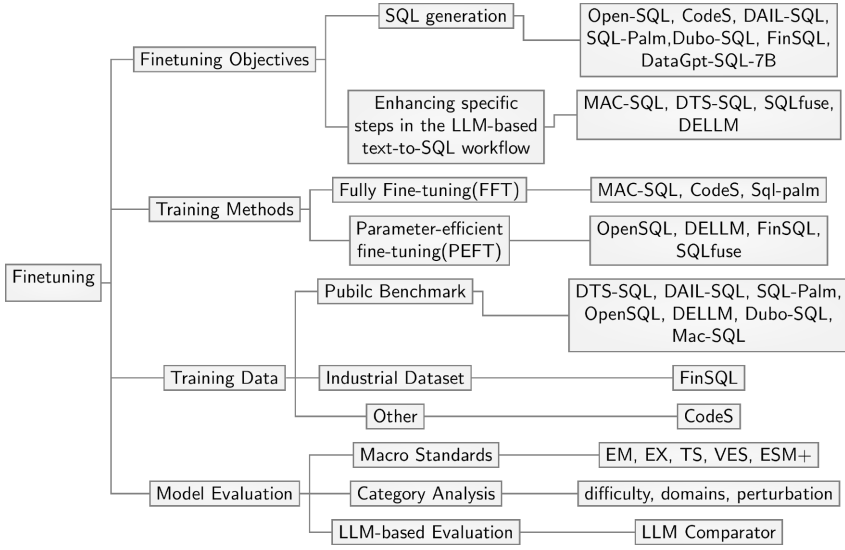


Fig. 6. Taxonomy of fine-tuning in Text-to-SQL.

We will structure this section into four parts, namely, fine-tuning objectives, training methods, and training data. Figure 6 presents related works classified in accordance with the aforementioned four parts.

5.1 Fine-tuning Objectives

Typically, the fine-tuning objective in the context of LLM-based text-to-SQL is SQL generation. That is, given the question representation and database information in specifically designed styles, models try to infer the correct SQL query. Examples of such approaches include Open-SQL [17], CodeS [59], DAIL-SQL [35], SQL-Palm [100], Dubo-SQL [105], FinSQL [134], and DataGpt-SQL-7B [116].

In addition to the aforementioned SQL generation objective, several studies [42, 86, 107, 138] utilize fine-tuning to enhance the performance of different steps in the LLM-based text-to-SQL workflow. MAC-SQL [107] designed a three-agent architecture comprising a selector that handles schema linking, a decomposer responsible for question decomposition and SQL generation, and a refiner that deals with SQL correction. In addition to employing GPT-4 [1] to act as the three agents, MAC-SQL [107] also explored the use of fine-tuned LLMs to perform the roles of the three agents. Both DTS-SQL [86] and SQLfuse [138] explored fine-tuning LLMs for schema linking and sql generation. Creatively, DELLM [42] fine-tuned an LLM to generate domain knowledge based on given questions and database information. This generated domain knowledge would then be integrated into prompts to assist in knowledge-intensive SQL generation.

5.2 Training Methods

There are two streams of training methods in LLM finetuning, namely, **fully fine-tuning (FFT)** and **parameter-efficient fine-tuning (PEFT)** [24, 45, 56, 66, 67, 70]. FFT treats all parameters in LLMs as trainable parameters while PEFT only tunes a small number of parameters, which improves training efficiency and reduces training costs. Also, according to research in [46], PEFT is less prone to catastrophic forgetting compared to FFT during use, which again proves its superiority.

In the context of LLM-based text-to-SQL, finetuning adopts both FFT and PEFT and among PEFT methods, LoRA [45] and QLoRA [24] are most popular methods. LoRA [45], short for Low-Rank Adaptation, freezes the pretrained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture. In this way, LoRA significantly reduces the number of trainable parameters for downstream tasks. QLoRA [24] is a finetuning method developed based on LoRA. Through a series of innovative efforts such as introducing 4-bit NormalFloat, double quantization, and Paged Optimizers, QLoRA refined the LoRA method, reducing memory usage while maintaining performance. The examples of employing FFT in the context of LLM-based text-to-SQL are Mac-SQL [107], CodeS [59], and Sql-palm [100] while the examples of employing LORA [45] and QLoRA [24] are OpenSQL [17], DELLM [42], FinSQL [134], and SQLfuse [138].

5.3 Training Data

Due to the goal of exploring fine-tuning effect, most research articles [17, 35, 42, 86, 100, 105, 107] on finetuning in the context of LLM-based text-to-SQL utilize the training set of open-source Text-to-SQL benchmarks directly. Among them, DTS-SQL [86], DAIL-SQL [35], and SQL-Palm [100] finetuned LLMs using Spider [129] training set. OpenSQL [17], DELLM [42], and Dubo-SQL [105] using BIRD [60] training set. Mac-SQL [107] conducts experiments using both datasets. The training data may go through preprocessing (for instance, schema linking and knowledge supplementation) to be transformed into a designed representation format.

As an example of industrial article, FinSQL [134] takes into account the unique characteristics of databases in financial applications. As public benchmarks do not consider these characteristics, FinSQL [134] constructed the BULL benchmark. This benchmark is collected from the practical financial analysis business of Hundsun Technologies Inc. and includes databases for funds, stocks, and the macro economy.

In addition to the aforementioned method, CodeS [59] sheds some light on how to construct fine-tuning datasets to enhance general text-to-SQL capability and how to construct fine-tuning datasets to adaptively transfer to databases within a new domain. For enhancing general text-to-SQL capability, CodeS [59] collected 21.5 GB of data, consisting of SQL-related data (11 GB), NL-to-code data (6 GB), and NL-related data (4.5 GB), with the aim of enhancing both SQL generation capabilities and NL comprehension. For adaptively transferring to databases within a new domain, CodeS [59] collects a few genuine user queries, manually annotate corresponding SQL queries, and leverage GPT-3.5 to produce a wider set of (question, SQL) pairs in a few-shot manner, ensuring user-oriented authenticity. Meanwhile, CodeS [59] utilizes SQL templates and their question templates from text-to-SQL benchmarks. By plugging in tables, columns, and values from the databases of new domains into the templates, CodeS [59] generates a diverse set of (question, SQL) pairs.

5.4 Model Evaluation

After fine-tuning the model, an important step is to compare the before and after results to understand the changes in the model's performance. In the context of text-to-SQL, the common approach is to conduct a comprehensive metric analysis on the test set, such as calculating the accuracy rates of EM and EX metrics, before and after fine-tuning. However, beyond these metrics, a detailed analysis can uncover more in-depth insights. For instance, we can categorize NL queries into several groups by considering their difficulty levels [129], domains [60], perturbation types [12], and so on. Additionally, we can classify the error types of the output results for analysis [17, 25, 85, 137]. Moreover, the emergence of LLM also opens up new possibilities for model evaluation analysis. LLM

Comparator [48] proposed a system that can visualize and evaluate the differences between two models, analyzing the differences from the “when, why, how” dimensions. Although the method of LLM Comparator [48] is not specifically designed for the text2sql task, we can apply the query classification methods (e.g., difficulties and domains) and macro standards (e.g., EM, EX, TS [143], VES [60], and ESM+ [5]) to the evaluation system of LLM Comparator [48], thereby migrating the evaluation system to the text2sql field.

5.5 Current Situation

Due to considerations of enterprise privacy information and data security, industrial text-to-SQL is more suitable for adopting the fine-tuning method. However, compared to prompting approaches, fine-tuning approaches have been relatively under-explored in the research community. Firstly, due to the superior performance of closed-source models like GPT series [1], most methods centered on closed-source models can achieve better performance. Coupled with the low cost of API calls, the research on fine-tuning open-source models has not received as much attention as the prompt engineering method (this can be seen from the length of the prompt engineering and fine-tuning sections in this survey), which to some extent has led to a lag in the research of fine-tuning open-source model methods. Secondly, the prompt engineering method has more innovation points at the algorithmic level. In contrast, the fine-tuning method has fewer innovation points at the algorithmic level and more relies on progress in training techniques, base models, and the quality of training data. This also leads to a lower popularity of the fine-tuning for text-to-SQL method.

Key takeaways

- Fine-tuning can be employed to enhance the performance of various steps in the LLM-based text-to-SQL workflow.
- Most research works prefer PEFT over FFT due to PEFT’s superior training efficiency and lower training costs.
- Open benchmarks lack the characteristics of databases in industrial applications.
- Leveraging LLMs to generate datasets holds significant potential.

6 Model

Whether prompt engineering or fine-tuning is used, the choice of base LLM is important as the implementation basis of text-to-SQL tasks is based on it. LLMs used in text-to-SQL tasks are usually based on Transformer’s architecture [106] and adopt a decoder-only pattern suitable for open-generation tasks. We will primarily classify LLMs based on whether they are open-source. We will introduce the basic information of typical models, present the application of these models in the text-to-SQL domain, and finally analyze the development trend of basic LLMs and provide suggestions on their usage in Text2SQL tasks.

6.1 Base Model

After our systematic investigation, the existing LLM-based text-to-SQL works mainly use 12 closed-source LLMs and 16 open-source LLMs as base models. Some articles conduct experiments on multiple models simultaneously to compare the effects of the same approach on different models. Here we count all the models related to the approaches and experiments, rather than just the best-performing model for each method. Figure 7 shows the frequency of use of these models in the

article's methods and experiments. The data is collected by identifying the specific LLMs employed for SQL generation in the experiments of each surveyed article, including [3, 4, 8, 10, 11, 16–18, 25, 27, 35, 37, 38, 42, 54, 61, 65, 69, 72, 78, 82, 84–86, 90, 97, 99, 103, 107, 108, 111, 117, 118, 120, 131, 135–138], all published prior to the latest revision of this survey.

6.1.1 Closed-source Models. The main advantage of closed-source models lies in their large-scale pre-trained corpus and the huge parameter size. Taking GPT-4 [1] as an example, it is pre-trained on about 13T tokens (including text data and code data) and has about 1.8 trillion (1,800 B) parameters across 120 layers. GPT-4 is also the most common base model in LLM-based text-to-SQL work [16, 27, 35, 61, 85, 90, 99, 103, 107, 117, 120, 135, 137, 138]. Although slightly inferior to GPT-4, GPT-3.5 series also act as a strong baseline model for most text-to-SQL methods [4, 16, 17, 25, 27, 35, 69, 90, 99, 102, 103, 111, 117, 120, 135–137]. As a descendant of GPT-3, the closed-source LLM CodeX series [14] which focuses on code generation is pre-trained on billions of lines of code and has more than 14K Python code memory. CodeX has been used in many text-to-SQL approaches [3, 16, 18, 39, 78, 85, 102, 137] due to its expertise in many programming languages. In addition, text-davinci-003 model and PaLM-2 [2] are also used by some methods [4, 35, 37, 69, 137] in the text-to-SQL domain. Since closed-source large models have large parameters and are difficult to deploy independently, most methods or experiments in the literature work use API calls to access these models. However, as of the time of writing this article, there is no closed-source LLM specifically for SQL code generation. There are two possible reasons: First, SQL involves privacy information in different business data processing, so it is difficult to directly obtain a certain scale of pre-training data; second, the generalization of the language model specifically generated for SQL may not be as good as the general code generation model. This part can be a direction for future exploration.

6.1.2 Open-source Models. Unlike closed-source models, open-source models are often deployed or fine-tuned in the private domain by users with certain hardware support due to their appropriate parameter scale. Usually, open-source LLMs have versions of different model sizes. Generally speaking, the more model parameters, the stronger the model generation capability [35]. The Llama 3 series [26] commonly used in text-to-SQL is divided into two types: 8 B parameter size and 70 B parameter size. In addition, the Code Llama series [91] has several parameter sizes of 7 B, 13 B, 34 B, and 70 B, and its performance in code generation and completion in languages such as Python and C++ exceeds that of the non-dedicated model Llama 2 [35]. Deepseek [21] and QWen [124] also use version numbers to indicate their different model types with various parameter sizes. Among the open-source large models, SQLCoder [98] is a model family dedicated to text-to-SQL tasks. Its 70 B parameter version and 15 B parameter version respectively surpass the performance of GPT-4 and GPT-3.5-turbo on a new dataset based on SPIDER [129] with novel schemas not seen in training. Most text-to-SQL methods based on prompt engineering that use open-source models often view them as baselines for experiments. For example, DAIL-SQL [35] compared Llama, Llama 2, Coda Llama, Alpaca [104], and Vicuna models [142] in the experiment. The latter two are general models fine-tuned based on Llama 7B and Llama 13B, respectively. PET-SQL [61] compared Code Llama, SQLCoder, InternLM [9], and SenseChat [95] in the experiment, while DEA-SQL [120] selected Llama 2, Code Llama, and WizardCoder [121] as the base models of its method. In the work using the finetune method, LEVER [78] fine-tuned InCoder [30] and CodeGen [79]. Open-SQL [17] fine-tuned Llama 2 and Code Llama in the text-to-SQL domain. The effect of Code Llama after fine-tuning on the BIRD dataset has exceeded that of the closed-source LLM GPT-4. CHESS [103] uses the Llama 3 70B model and the fine-tuned DeepSeek Coder [40] model,

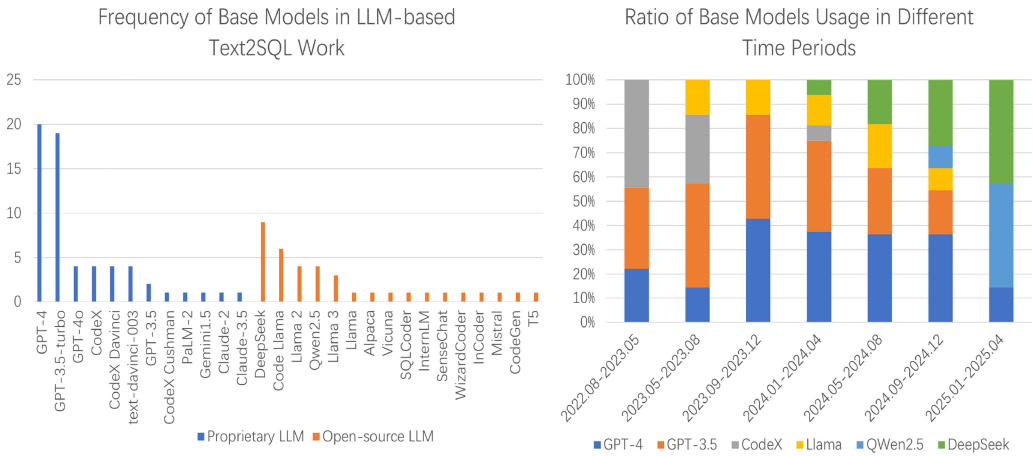


Fig. 7. Frequency of LLM usage in Text2SQL research works from August 2022 to April 2025 (left). Trend of base model series usage in text-to-SQL tasks (right).

and its effect on the BIRD-dev dataset is comparable to that of the methods based on closed-source LLMs.

6.2 Trend

Figure 7 shows the trend of the usage frequency of the base models in the LLM-based text-to-SQL methods and experiments over time, as well as the time when each base model series was launched. It can be found that within the scope of closed-source LLMs, the emergence of GPT-4 [1] (in March 2023) has made most of the work prefer to use the GPT series models rather than CodeX series; within the scope of open-source LLMs, most of the work will choose Deepseek [21], Llama, Code Llama, and QWen [124] series of base models. As time went by, the frequency of use of closed-source models increased in 2023 but decreased in 2024, while the open-source models were the opposite. One possible reason is that the growing parameter size in open-source models makes them improve task performance while maintaining privacy and fine-tuning abilities. In the future, as the cost of LLMs decreases, the most powerful closed-sourced LLM and the most powerful open-sourced LLM will become dominant in text-to-SQL tasks.

Key takeaways

- In current text-to-SQL tasks, the advantage of closed-source base models is that they are ready to use and have powerful code-generation capabilities. A combination of closed-source models and prompt engineering methods is suitable for users with insufficient hardware support.
- The advantages of using open-source base models are independent deployment, privacy, and the ability to fine-tune domains. Users with hardware support can combine open-source models with fine-tuning for in-domain text-to-SQL tasks.
- Existing LLM-based text-to-SQL work favors the GPT series among closed-source models and the DeepSeek, Llama, Code Llama, and Qwen series among open-source models.
- Until now, open-source models have been used more and more frequently and their usage is equally matched to that of closed-source models in the research aspect.

7 Analysis

To further investigate the practical value and applicability of each method, we conduct a comparative analysis of experimental results from existing studies.

7.1 Benchmark Selection

We evaluate performance on three widely used benchmarks in Text-to-SQL research: Spider 1.0 [129], BIRD [60], and Spider 2.0 [55], each representing key challenges in the field.

Spider 1.0 [129] remains the most established benchmark, serving as the primary dataset for evaluating Text-to-SQL model performance. Its standardized schema and well-defined complexity levels make it a cornerstone for comparative studies. BIRD [60] introduces real-world challenges, such as handling noisy and incomplete database values, as well as incorporating external knowledge to bridge NL questions with database content. This dataset significantly narrows the gap between academic research and practical applications. Spider 2.0 [55] extends the scope to enterprise-level workflows, featuring complex multi-database environments (both cloud and local), diverse SQL dialects, and a broad range of operations—from data transformation to analytical queries.

Overall, these datasets demonstrate a clear trend toward greater realism, with SQL queries becoming more intricate and challenging, closely mirroring industrial use cases.

7.2 Evaluation Methodology

To ensure a fair comparison, we extract results from the official leaderboards of Spider1.0 [129], BIRD [60], and Spider 2.0 [55], adhering to the following selection criteria: 1. Only approaches leveraging LLMs are considered; 2. A maximum of two best-performing LLMs per method are included; 3. Methods based on models released prior to 2022 (pre-ChatGPT) are excluded to focus on recent advancements; 4. Methods relying on undocumented models (e.g., closed systems lacking technical details or API access) are disregarded; 5. Non-reproducible approaches (e.g., those without open-source code, published articles, or sufficient implementation details) are excluded to maintain fairness and practical utility.

7.3 Results

We adopt the EX metric and the results are shown in Figure 8.

Spider 1.0. All methods included in our analysis represent typical LLM-based Text-to-SQL approaches, with most achieving pass rates exceeding 80%. This demonstrates that modern LLMs have effectively solved basic Text-to-SQL tasks on the Spider benchmark, confirming their competence in handling fundamental semantic parsing challenges.

BIRD. At the model level, both powerful closed-source and open-source LLMs combined with prompting strategies, as well as smaller open-source LLMs enhanced through fine-tuning, demonstrate competitive, SOTA performance. This suggests that the performance gap between closed-source and open-source models has narrowed significantly and compact models enhanced via fine-tuning can show strong potential to match the capabilities of much larger LLMs.

At the methodological level, our comparison focuses on approaches using the same underlying LLMs to ensure fairness. Although limited experimental data makes it difficult to pinpoint the exact reasons for performance differences between methods, several distinctive characteristics can be observed in the most effective approaches. In particular, we highlight OpenSearch-SQL [119] (the top-performing method under GPT-4o and second-best under GPT-4), Distillery [74] (second-best under GPT-4o), and MCS-SQL [54] (top-performing under GPT-4).

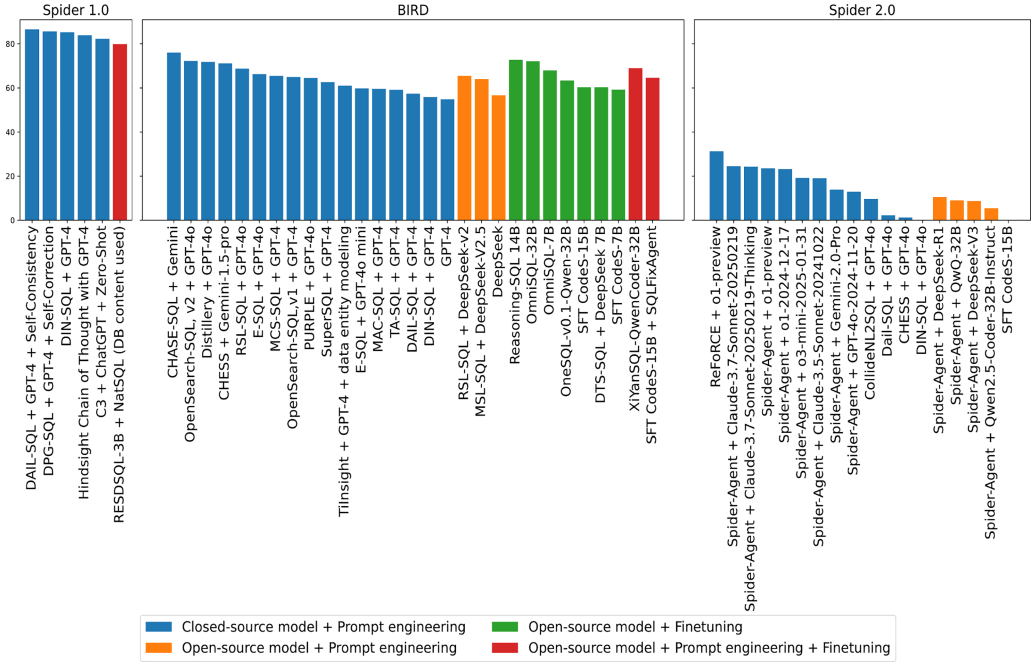


Fig. 8. Execution accuracy of different combinations of methods and LLMs on the test sets of the Spider 1.0 [129], BIRD [60], and Spider 2.0 [55] benchmarks.

Characteristic 1: Inference-time Scaling. These top-performing methods leverage advanced inference-time strategies. OpenSearch-SQL [119] incorporates multi-SQL generation, correction mechanisms, and self-consistency checks. Distillery [74] employs iterative correction combined with self-consistency, while MCS-SQL [54] adopts multiple prompts for schema linking and SQL generation, thereby exploring a broader search space for potential answers. In contrast, baseline methods such as vanilla DIN-SQL [85] rely on single-step SQL correction, and vanilla DAIL-SQL [35] lacks any correction or self-consistency components.

Characteristic 2: Enhanced Schema Linking. Distillery [74] investigates the impact of schema linking errors and observes that modern LLMs are increasingly tolerant of irrelevant columns. Consequently, it recommends including as many columns as possible when context length permits. MCS-SQL [54], on the other hand, enhances schema linking through its inference-time scaling strategy, which broadens the exploration space and improves the likelihood of generating correct SQL queries.

Spider 2.0. The Spider 2.0 [55] evaluation provides valuable insights into the capabilities and limitations of LLM-based Text-to-SQL systems. The performance of Spider-Agent [55] across different LLMs highlights that, although open-source models have made significant progress, they still lag behind proprietary alternatives. The evaluation also shows that reasoning-enhanced models consistently deliver stronger performance. Furthermore, we observe that non-agent approaches significantly underperform compared to agent-based methods. This may be attributed to the complexity of real-world Text-to-SQL tasks, which often involve diverse SQL dialects, complex syntax and functions, nested columns, the integration of external knowledge, and the need to interpret ambiguous user requests. These challenges underscore the importance of agentic approaches, which allow LLMs to dynamically interact with databases, receive feedback, and iteratively plan

their actions. Notably, REFORCE [22], a recent agent-based method, outperforms standard Spider agents by incorporating advanced techniques such as table compression for context optimization, structured output formatting, iterative column exploration for better schema understanding, and a novel self-refinement pipeline that combines parallelized voting with CTE-based resolution. Despite these promising developments, it is important to note that the current SOTA method achieves only a 31.26% execution accuracy, underscoring a significant gap between academic advancements and real-world deployment.

Key takeaways

- LLMs excel on Spider 1.0, showing that foundational Text-to-SQL tasks are largely solved by modern models.
- Effective inference-time strategies and schema linking may significantly boost performance.
- Spider 2.0 reveals real-world gaps, where agent-based methods outperform others, but overall execution accuracy remains low, highlighting the need for more robust, deployable solutions.

8 Future Direction

Although LLM technology has significantly improved text-to-SQL tasks, there are still many difficulties in developing high-quality text-to-SQL parsers. In this section, we will first conduct a review of the error analysis of research approaches. Subsequently, we will introduce the challenges faced by existing solutions and some promising future development directions for LLM-based text-to-SQL tasks.

8.1 Error Analysis Overview

Some existing methods [17, 25, 85, 137] have conducted error analysis on the test results of SPIDER and BIRD datasets. Their error classification method is based on the components that produce errors in the generated SQL. There are five major categories: schema-linking, join, nested, group by, and others (Open-SQL [17] categorizes “group by” into others). Each major category also has subcategories such as wrong column, wrong table, and wrong condition. Among them, the average error rate in schema-linking is the highest, at 29%–49%, the error rate near *JOIN* is 21%–26%, and the error rate of *GROUP BY* and nested is less than 20%. This shows that schema-linking is still the module that needs to be improved most in the current text-to-SQL task. The join connection and syntax problems of data tables also have considerable challenges. In [68], the errors listed by DIN-SQL are classified more finely for SQL keywords, involving keywords such as *SELECT*, *FROM*, *WHERE*, *ORDER BY*, *GROUP BY*, and more fine-grained types such as error, redundancy, and insufficiency. In addition, when performing error analysis, MAC-SQL [107] found that the gold answer in the SPIDER [129] and BIRD [60] benchmarks also had 20%–30% errors, demonstrating the necessity of data cleaning.

8.2 Practical Challenges and Directions

8.2.1 Privacy Concern. ChatGPT and GPT-4 [1] demonstrate impressive capabilities, acknowledged as the most powerful LLMs. However, concerns over privacy issues often arise while using API call models in production scenarios, because of the potential leakage of private information through feeding prompts to the API. Private deployment of open-sourced LLM and fine-tuning it can serve as a solution to this problem while maintaining performance. However, the dirty data,

possible compromise in LLM's other capabilities, and catastrophic forgetfulness are blocking the performance gain of fine-tuning in production scenarios. Extracting high-quality training data may help handle the above private fine-tuning issue.

8.2.2 Complex Schema and Insufficient Benchmark. Table schemas are much more complex than current research benchmarks in many real-world text-to-SQL tasks. One of Microsoft's internal financial data warehouse [28] encompasses 632 tables with over 4,000 columns and 200 views with more than 7,400 columns. The text-to-SQL task on this database shows some prominent issues: difficulty in schema linking, diversion of LLM's attention due to a torrent of tokens, and long inference time. To address schema-linking challenges, recent advancements provide promising directions. Some approaches [22, 55] leverage agent-based methods, such as ReAct [127], to interactively explore database structures, while others [54] employ self-consistency techniques to reduce schema-linking errors. Beyond these methods, emerging developments in dynamic graph attention mechanisms [13] present promising opportunities. Specifically, adaptive graph-based representations can be constructed to model the relationships among tables, columns, and queries.

Current standard datasets can not model the complexity of the above Microsoft's warehouse. Spider [129] only contains simple table schemas. BIRD [60] emulates real-world scenarios, but the scale is far not enough. Spider 2.0 [55] provides a good example to incorporate common local or cloud database systems such as BigQuery and Snowflake, showing the potential direction of real-world usage of Text-to-SQL benchmarks.

8.2.3 Domain Knowledge. The training of LLMs on extensive corpora equips them with broad general knowledge, forming the basis for their strong text-to-SQL capabilities. However, real-world applications such as data analysis and batch processing demand domain-specific knowledge, including industry terminology, abbreviations, and jargon. Without such knowledge, LLMs often misinterpret or inaccurately respond to queries. Domain adaptation can be achieved through prompt engineering and fine-tuning, though both face significant challenges. Prompt engineering, often relying on RAG [36], requires a high-quality structured knowledge base. In practice, domain knowledge is typically embedded in noisy, unstructured documents, complicating knowledge base construction. Furthermore, similarity-based retrieval may introduce irrelevant information, degrading model performance. Fine-tuning, while embedding domain knowledge directly into model parameters, risks catastrophic forgetting and limits future adaptability; modifying or updating embedded knowledge often necessitates repeated, costly retraining.

8.2.4 Autonomous Agents. Building on the ReAct framework [127], it is now feasible to develop autonomous agents powered by LLMs. In such architectures, the LLM serves as the agent's core, orchestrating interactions with environments, humans, and other agents, while leveraging short- and long-term memory to iteratively refine task execution. The motivation for applying agents to text-to-SQL is from how humans naturally compose SQL: human SQL construction involves trial-and-error cycles of execution and revision, a workflow well-suited for agent-based modeling. Notably, REFORCE [22] and Spider 2.0 [55] are the first frameworks to explicitly adopt autonomous agents for text-to-SQL tasks. Results from the Spider 2.0 leaderboard show that traditional non-agent methods [35, 85, 103] underperform compared to these agent-based approaches, highlighting the potential of LLM-driven agents in this domain.

8.2.5 Data Governance. Two key limitations in current text-to-SQL datasets are ambiguity and semantic mismatch [28]. Ambiguity arises when multiple semantically distinct SQL queries yield valid answers to the same NL question, complicating the evaluation of model correctness. Semantic mismatch occurs when user queries reflect intents that cannot be answered by the underlying database, either partially or entirely. In the context of LLM-based text-to-SQL systems,

data governance offers three major benefits: (1) structuring domain knowledge enhances RAG effectiveness and improves alignment between NL queries and executable logic, thereby reducing ambiguity; (2) improving training data quality strengthens LLMs' domain-specific capabilities during fine-tuning; and (3) refining benchmark datasets to address semantic mismatch enables more robust evaluation of practical system performance.

9 Conclusion

This article offers a comprehensive review of the utilization of LLMs in Text-to-SQL tasks. We first enumerate classic benchmarks and evaluation metrics. For the two mainstream methods, prompt engineering and finetuning, we introduce a comprehensive taxonomy and offer practical insights into each subcategory. We present an overall analysis of the above methods and various models evaluated on well-known datasets and extract some insights. Finally, we discuss the challenges and future directions in this field.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774>
- [2] Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. arXiv:2305.10403. Retrieved from <https://arxiv.org/abs/2305.10403>
- [3] Aseem Arora, Shabbirhussain Bhaisaheb, Manasi Patwardhan, Lovekesh Vig, and Gautam Shroff. [n. d.]. A generic prompt for an LLM that enables NL-TO-SQL across domains and compositions. ([n. d.]).
- [4] Aseem Arora, Shabbirhussain Bhaisaheb, Manasi Patwardhan, Lovekesh Vig, and Gautam Shroff. 2023. Adapt and decompose: Efficient generalization of Text-to-SQL via domain adapted least-to-most prompting. arXiv:2308.02582. Retrieved from <https://arxiv.org/abs/2308.02582>
- [5] Benjamin Ascoli, Ram Kandikonda, and Jinho D. Choi. 2024. ESM+: Modern insights into perspective on Text-to-SQL evaluation in the age of large language models. arXiv:2407.07313. Retrieved from <https://arxiv.org/abs/2407.07313>
- [6] Ben Bogin, Matt Gardner, and Jonathan Berant. 2019. Representing schema structure with graph neural networks for text-to-SQL parsing. arXiv:1905.06241. Retrieved from <https://arxiv.org/abs/1905.06241>
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.
- [8] Hasan Alp Caferoğlu and Özgür Ulusoy. 2024. E-sql: Direct schema linking via question enrichment in text-to-sql. arXiv:2409.16751. Retrieved from <https://arxiv.org/abs/2409.16751>
- [9] Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin Chen, Xun Chen, Zehui Chen, Zhi Chen, Pei Chu, et al. 2024. InternLM2 Technical Report. arXiv:2403.17297. Retrieved from <https://arxiv.org/abs/2403.17297>
- [10] Zhenbiao Cao, Yuanlei Zheng, Zhihao Fan, Xiaojin Zhang, Wei Chen, and Xiang Bai. 2024. Rsl-sql: Robust schema linking in text-to-sql generation. arXiv:2411.00073. Retrieved from <https://arxiv.org/abs/2411.00073>
- [11] Shuaichen Chang and Eric Fosler-Lussier. 2023. How to prompt llms for text-to-sql: A study in zero-shot, single-domain, and cross-domain settings. arXiv:2305.11853. Retrieved from <https://arxiv.org/abs/2305.11853>
- [12] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Joseph Lilien, et al. 2023. Dr. spider: A diagnostic evaluation benchmark towards text-to-sql robustness. arXiv:2301.08881. Retrieved from <https://arxiv.org/abs/2301.08881>
- [13] Deng Chen, Weiwei Zhang, and Zuohua Ding. 2025. Embedding dynamic graph attention mechanism into Clinical Knowledge Graph for enhanced diagnostic accuracy. *Expert Systems with Applications* 267 (2025), 126215.
- [14] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>
- [15] Wenhui Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyu Zhou, and William Yang Wang. 2020. TabFact: A Large-scale Dataset for Table-based Fact Verification. arXiv:1909.02164. Retrieved from <https://arxiv.org/abs/1909.02164>
- [16] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. arXiv:2304.05128. Retrieved from <https://arxiv.org/abs/2304.05128>

- [17] Xiaojun Chen, Tianle Wang, Tianhao Qiu, Jianbin Qin, and Min Yang. 2024. Open-SQL Framework: Enhancing Text-to-SQL on Open-source Large Language Models. arXiv:2405.06674. Retrieved from <https://arxiv.org/abs/2405.06674>
- [18] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, et al. 2022. Binding language models in symbolic languages. arXiv:2210.02875. Retrieved from <https://arxiv.org/abs/2210.02875>
- [19] DongHyun Choi, Myeong Cheol Shin, EungGyun Kim, and Dong Ryeol Shin. 2021. Ryansql: Recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases. *Computational Linguistics* 47, 2 (2021), 309–332.
- [20] Sumit Kumar Dam, Choong Seon Hong, Yu Qiao, and Chaoning Zhang. 2024. A complete survey on LLM-based AI chatbots. arXiv:2406.16937. Retrieved from <https://arxiv.org/abs/2406.16937>
- [21] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiron Ma, Peiyi Wang, et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. Retrieved from <https://arxiv.org/abs/2501.12948>
- [22] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. 2025. ReForCE: A Text-to-SQL agent with self-refinement, format restriction, and column exploration. arXiv:2502.00675. Retrieved from <https://arxiv.org/abs/2502.00675>
- [23] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. 2020. Structure-grounded pretraining for text-to-sql. arXiv:2010.12773. Retrieved from <https://arxiv.org/abs/2010.12773>
- [24] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2024).
- [25] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Jinshu Lin, Dongfang Lou, et al. 2023. C3: Zero-shot text-to-sql with chatgpt. arXiv:2307.07306. Retrieved from <https://arxiv.org/abs/2307.07306>
- [26] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. arXiv:2407.21783. Retrieved from <https://arxiv.org/abs/2407.21783>
- [27] Yuankai Fan, Zhenying He, Tonghui Ren, Can Huang, Yinan Jing, Kai Zhang, and X. Sean Wang. 2024. Metasql: A Generate-then-Rank Framework for Natural Language to SQL Translation. arXiv:2402.17144. Retrieved from <https://arxiv.org/abs/2402.17144>
- [28] Avriella Floratou and Fotis Psallidas. 2024. NL2SQL is a solved problem... Not! In *Proceedings of the Conference on Innovative Data Systems Research*.
- [29] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30 (2020), 681–694.
- [30] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. [n. d.]. InCoder: A generative model for code infilling and synthesis. arXiv:2204.05999. Retrieved from <https://arxiv.org/abs/2204.05999>
- [31] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of llm inference using lookahead decoding. arXiv:2402.02057. Retrieved from <https://arxiv.org/abs/2402.02057>
- [32] Yujian Gan, Xinyun Chen, Qiuping Huang, and Matthew Purver. 2022. Measuring and improving compositional generalization in text-to-sql via component alignment. arXiv:2205.02054. Retrieved from <https://arxiv.org/abs/2205.02054>
- [33] Yujian Gan, Xinyun Chen, and Matthew Purver. 2021. Exploring underexplored limitations of cross-domain text-to-SQL generalization. arXiv:2109.05157. Retrieved from <https://arxiv.org/abs/2109.05157>
- [34] Yujian Gan, Xinyun Chen, Jinxia Xie, Matthew Purver, John R. Woodward, John Drake, and Qiaofu Zhang. 2021. Natural SQL: Making SQL easier to infer from natural language specifications. arXiv:2109.05153. Retrieved from <https://arxiv.org/abs/2109.05153>
- [35] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-sql empowered by large language models: A benchmark evaluation. arXiv:2308.15363. Retrieved from <https://arxiv.org/abs/2308.15363>
- [36] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997. Retrieved from <https://arxiv.org/abs/2312.10997>
- [37] Chunxi Guo, Zhiliang Tian, Jintao Tang, Shasha Li, Zhihua Wen, Kaixuan Wang, and Ting Wang. 2023. Retrieval-augmented GPT-3.5-based Text-to-SQL Framework with Sample-aware Prompting and Dynamic Revision Chain. arXiv:2307.05074. Retrieved from <https://arxiv.org/abs/2307.05074>

- [38] Chunxi Guo, Zhiliang Tian, Jintao Tang, Pancheng Wang, Zhihua Wen, Kang Yang, and Ting Wang. 2023. A case-based reasoning framework for adaptive prompting in cross-domain text-to-sql. arXiv:2304.13301. Retrieved from <https://arxiv.org/abs/2304.13301>
- [39] Chunxi Guo, Zhiliang Tian, Jintao Tang, Pancheng Wang, Zhihua Wen, Kang Yang, and Ting Wang. 2023. Prompting GPT-3.5 for Text-to-SQL with de-semanticization and skeleton retrieval. In *Proceedings of the Pacific Rim International Conference on Artificial Intelligence*. Springer, 262–274.
- [40] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the large language model meets programming-the rise of code intelligence. arXiv:2401.14196. Retrieved from <https://arxiv.org/abs/2401.14196>
- [41] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.
- [42] Zijin Hong, Zheng Yuan, Hao Chen, Qinggang Zhang, Feiran Huang, and Xiao Huang. 2024. Knowledge-to-SQL: Enhancing SQL generation with data expert LLM. In *Findings of the Association for Computational Linguistics ACL 2024*. 10997–11008.
- [43] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2024. Next-generation database interfaces: A survey of llm-based text-to-sql. arXiv:2406.08426. Retrieved from <https://arxiv.org/abs/2406.08426>
- [44] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large language models for software engineering: A systematic literature review. arXiv:2308.10620. Retrieved from <https://arxiv.org/abs/2308.10620>
- [45] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. arXiv:2106.09685. Retrieved from <https://arxiv.org/abs/2106.09685>
- [46] Joel Jang, Seungone Kim, Seonghyeon Ye, Doyoung Kim, Lajanugen Logeswaran, Moontae Lee, Kyungjae Lee, and Minjoon Seo. 2023. Exploring the benefits of training expert language models over instruction tuning. In *Proceedings of the International Conference on Machine Learning*. PMLR, 14702–14729.
- [47] Dipendra Jha, Logan Ward, Zijiang Yang, Christopher Wolverton, Ian Foster, Wei-keng Liao, Alok Choudhary, and Ankit Agrawal. 2019. IRNet: A general purpose deep residual regression framework for materials discovery. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2385–2393.
- [48] Minsuk Kahng, Ian Tenney, Mahima Pushkarna, Michael Xieyang Liu, James Wexler, Emily Reif, Krystal Kallarakal, Minsuk Chang, Michael Terry, and Lucas Dixon. 2024. LLM comparator: Visual analytics for side-by-side evaluation of large language models. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–7.
- [49] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. arXiv:2001.08361. Retrieved from <https://arxiv.org/abs/2001.08361>
- [50] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the naaCL-HLT*. Minneapolis, Minnesota.
- [51] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. arXiv:2210.02406. Retrieved from <https://arxiv.org/abs/2210.02406>
- [52] Mayank Kothiyari, Dhruva Dhingra, Sunita Sarawagi, and Soumen Chakrabarti. 2023. CRUSH4SQL: Collective retrieval using schema hallucination for Text2SQL. arXiv:2311.01173. Retrieved from <https://arxiv.org/abs/2311.01173>
- [53] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. 2021. KaggleDBQA: Realistic evaluation of text-to-SQL parsers. arXiv:2106.11455. Retrieved from <https://arxiv.org/abs/2106.11455>
- [54] Dongjun Lee, Choongwon Park, Jaehyuk Kim, and Heesoo Park. 2025. MCS-SQL: Leveraging multiple prompts and multiple-choice selection for Text-to-SQL generation. In *Proceedings of the 31st International Conference on Computational Linguistics*. 337–353.
- [55] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2024. Spider 2.0: Evaluating language models on real-world enterprise Text-to-SQL workflows. arXiv:2411.07763. Retrieved from <https://arxiv.org/abs/2411.07763>
- [56] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. arXiv:2104.08691. Retrieved from <https://arxiv.org/abs/2104.08691>
- [57] Fei Li and Hosagrahar V. Jagadish. 2014. Constructing an interactive natural language interface for relational databases. *Proceedings of the VLDB Endowment* 8, 1 (2014), 73–84.
- [58] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 13067–13075.

- [59] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. CodeS: Towards Building Open-source Language Models for Text-to-SQL. arXiv:2402.16347. Retrieved from <https://arxiv.org/abs/2402.16347>
- [60] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? A big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).
- [61] Zhishuai Li, Xiang Wang, Jingjing Zhao, Sun Yang, Guoqing Du, Xiaoru Hu, Bin Zhang, Yuxiao Ye, Ziyue Li, Rui Zhao, et al. 2024. PET-SQL: A prompt-enhanced two-stage Text-to-SQL framework with cross-consistency. arXiv:2403.09732. Retrieved from <https://arxiv.org/abs/2403.09732>
- [62] Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. Bridging textual and tabular data for cross-domain Text-to-SQL semantic parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Trevor Cohn, Yulan He, and Yang Liu (Eds.), Association for Computational Linguistics, Online, 4870–4888. DOI: <https://doi.org/10.18653/v1/2020.findings-emnlp.438>
- [63] Aiwei Liu, Xuming Hu, Li Lin, and Lijie Wen. 2022. Semantic enhanced Text-to-SQL parsing via iteratively learning schema linking graph. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD'22)*. Association for Computing Machinery, New York, NY, USA, 1021–1030. DOI: <https://doi.org/10.1145/3534678.3539294>
- [64] Aiwei Liu, Xuming Hu, Lijie Wen, and Philip S. Yu. 2023. A comprehensive evaluation of ChatGPT's zero-shot Text-to-SQL capability. arXiv:2303.13547. Retrieved from <https://arxiv.org/abs/2303.13547>
- [65] Geling Liu, Yunzhi Tan, Ruichao Zhong, Yuanzhen Xie, Lingchen Zhao, Qian Wang, Bo Hu, and Zang Li. 2025. Solid-SQL: Enhanced schema-linking based in-context learning for robust Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics*. 9793–9803.
- [66] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin A. Raffel. 2022. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *Advances in Neural Information Processing Systems* 35 (2022), 1950–1965.
- [67] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2021. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. arXiv:2110.07602. Retrieved from <https://arxiv.org/abs/2110.07602>
- [68] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A survey of NL2SQL with large language models: Where are we, and where are we going? arXiv:2408.05109. Retrieved from <https://arxiv.org/abs/2408.05109>
- [69] Xiping Liu and Zhao Tan. 2023. Divide and prompt: Chain of thought prompting for text-to-sql. arXiv:2304.11556. Retrieved from <https://arxiv.org/abs/2304.11556>
- [70] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. 2023. GPT understands, too. *AI Open* (2023).
- [71] Yinhan Liu, MyLe Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. arXiv:1907.11692. Retrieved from <https://arxiv.org/abs/1907.11692>
- [72] Ruilin Luo, Liyuan Wang, Binghuai Lin, Zicheng Lin, and Yujiu Yang. 2024. PTD-SQL: Partitioning and targeted drilling with LLMs in Text-to-SQL. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 3767–3799.
- [73] Qin Lyu, Kaushik Chakrabarti, Shobhit Hathi, Souvik Kundu, Jianwen Zhang, and Zheng Chen. 2020. Hybrid ranking network for text-to-sql. arXiv:2008.04759. Retrieved from <https://arxiv.org/abs/2008.04759>
- [74] Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, and Amine Mhedhbi. 2024. The death of schema linking? Text-to-SQL in the age of well-reasoned language models. arXiv:2408.07702. Retrieved from <https://arxiv.org/abs/2408.07702>
- [75] Youssef Mellah, Abdelkader Rhouati, El Hassane Ettifouri, Toumi Bouchentouf, and Mohammed Ghaouth Belkasm. 2021. COMBINE: A pipeline for SQL generation from natural language. In *Proceedings of the Advances in Computing and Data Sciences: 5th International Conference, ICACDS 2021, Nashik, India, April 23–24, 2021, Revised Selected Papers, Part II* 5. Springer, 97–106.
- [76] Qingkai Min, Yuefeng Shi, and Yue Zhang. 2019. A pilot study for Chinese SQL semantic parsing. arXiv:1909.13293. Retrieved from <https://arxiv.org/abs/1909.13293>
- [77] Linyong Nan, Yilun Zhao, Weijin Zou, Narutatsu Ri, Jaesung Tae, Ellen Zhang, Arman Cohan, and Dragomir Radev. 2023. Enhancing text-to-SQL capabilities of large language models: A study on prompt design strategies. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 14935–14956.
- [78] Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen tau Yih, Sida I. Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. arXiv:2302.08468. Retrieved from <https://arxiv.org/abs/2302.08468>

- [79] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. arXiv:2203.13474. Retrieved from <https://arxiv.org/abs/2203.13474>
- [80] OpenAI. 2024. Introducing OpenAI o1-Preview. Retrieved from <https://openai.com/index/introducing-openai-o1-preview/>
- [81] Liangming Pan, Michael Saxon, Wenda Xu, Deepak Nathani, Xinyi Wang, and William Yang Wang. 2023. Automatically Correcting Large Language Models: Surveying the Landscape of Diverse Self-Correction Strategies. arXiv:2308.03188. Retrieved from <https://arxiv.org/abs/2308.03188>
- [82] Wenqi Pei, Hailing Xu, Hengyuan Zhao, Shizheng Hou, Han Chen, Zining Zhang, Pingyi Luo, and Bingsheng He. 2025. Feather-SQL: A lightweight NL2SQL framework with dual-model collaboration paradigm for small language models. arXiv:2503.17811. Retrieved from <https://arxiv.org/abs/2503.17811>
- [83] Xinyu Pi, Bing Wang, Yan Gao, Jiaqi Guo, Zhoujun Li, and Jian-Guang Lou. 2022. Towards robustness of text-to-SQL models against natural and realistic adversarial table perturbation. arXiv:2212.09994. Retrieved from <https://arxiv.org/abs/2212.09994>
- [84] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O Arik. 2024. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. arXiv:2410.01943. Retrieved from <https://arxiv.org/abs/2410.01943>
- [85] Mohammadreza Pourreza and Davood Rafiei. 2024. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems* 36 (2024).
- [86] Mohammadreza Pourreza and Davood Rafiei. 2024. DTS-SQL: Decomposed Text-to-SQL with small large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 8212–8220.
- [87] Jiexing Qi, Jingyao Tang, Ziwei He, Xiangpeng Wan, Yu Cheng, Chenghu Zhou, Xinbing Wang, Quanshi Zhang, and Zhouhan Lin. 2022. RASAT: Integrating relational structures into pretrained Seq2Seq model for Text-to-SQL. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 3215–3229. DOI : <https://doi.org/10.18653/v1/2022.emnlp-main.211>
- [88] Bowen Qin, Binyuan Hui, Lihan Wang, Min Yang, Jinyang Li, Binhua Li, Ruiying Geng, Rongyu Cao, Jian Sun, Luo Si, et al. 2022. A survey on text-to-sql parsing: Concepts, methods, and future directions. arXiv:2208.13629. Retrieved from <https://arxiv.org/abs/2208.13629>
- [89] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the Text-to-SQL capabilities of large language models. arXiv:2204.00498. Retrieved from <https://arxiv.org/abs/2204.00498>
- [90] Tonghui Ren, Yuankai Fan, Zhenying He, Ren Huang, Jiaqi Dai, Can Huang, Yinan Jing, Kai Zhang, Yifan Yang, and X. Sean Wang. 2024. PURPLE: Making a Large Language Model a Better SQL Writer. arXiv:2403.20014. Retrieved from <https://arxiv.org/abs/2403.20014>
- [91] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. arXiv:2308.12950. Retrieved from <https://arxiv.org/abs/2308.12950>
- [92] Ohad Rubin and Jonathan Berant. 2020. SmBoP: Semi-autoregressive bottom-up semantic parsing. arXiv:2010.12412. Retrieved from <https://arxiv.org/abs/2010.12412>
- [93] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. arXiv:2109.05093. Retrieved from <https://arxiv.org/abs/2109.05093>
- [94] Jaydeep Sen, Chuan Lei, Abdul Quamar, Fatma Ozcan, Vasilis Efthymiou, Ayushi Dalmia, Greg Stager, Ashish Mittal, Diptikalyan Saha, and Karthik Sankaranarayanan. 2020. ATHENA++: Natural language querying for complex nested SQL queries. *Proceedings of the VLDB Endowment* 13, 11 (2020), 2747–2759.
- [95] SenseTime. 2024. SenseChat. Retrieved from <https://platform.sensenova.cn/#/doc?path=/chat/ChatCompletions/ChatCompletions.md>
- [96] Burr Settles. 2009. Active learning literature survey. (2009).
- [97] Lei Sheng, Shuai-Shuai Xu, and Wei Xie. 2025. BASE-SQL: A powerful open source Text-To-SQL baseline approach. arXiv:2502.10739. Retrieved from <https://arxiv.org/abs/2502.10739>
- [98] Rishabh Srivastava and Wendy Aw. 2023. Defog SQLCoder. Retrieved from <https://github.com/defog-ai/sqlcoder>
- [99] Guanghu Sui, Zhishuai Li, Ziyue Li, Sun Yang, Jingqing Ruan, Hangyu Mao, and Rui Zhao. 2023. Reboost large language model-based Text-to-SQL, Text-to-Python, and text-to-function-with real applications in traffic domain. arXiv:2310.18752. Retrieved from <https://arxiv.org/abs/2310.18752>
- [100] Ruoxi Sun, Sercan O Arik, Hootan Nakhost, Hanjun Dai, Rajarishi Sinha, Pengcheng Yin, and Tomas Pfister. 2023. Sql-palm: Improved large language model adaptation for text-to-sql. arXiv:2306.00739. Retrieved from <https://arxiv.org/abs/2306.00739>

- [101] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems* 27 (2014).
- [102] Chang-Yu Tai, Ziru Chen, Tianshu Zhang, Xiang Deng, and Huan Sun. 2023. Exploring chain of thought style prompting for Text-to-SQL. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 5376–5393.
- [103] Shayan Talaie, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. CHES: Contextual Harnessing for Efficient SQL Synthesis. arXiv:2405.16755. Retrieved from <https://arxiv.org/abs/2405.16755>
- [104] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA Model. Retrieved from https://github.com/tatsu-lab/stanford_alpaca
- [105] Dayton G. Thorpe, Andrew J. Duberstein, and Ian A. Kinsey. 2024. Dubo-SQL: Diverse Retrieval-Augmented Generation and Fine-tuning for Text-to-SQL. arXiv:2404.12560. Retrieved from <https://arxiv.org/abs/2404.12560>
- [106] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).
- [107] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Qian-Wen Zhang, Zhao Yan, and Zhoujun Li. 2023. Mac-sql: Multi-agent collaboration for text-to-sql. arXiv:2312.11242. Retrieved from <https://arxiv.org/abs/2312.11242>
- [108] Dingzirui Wang, Longxu Dou, Xuanliang Zhang, Qingfu Zhu, and Wanxiang Che. 2024. Improving Demonstration Diversity by Human-Free Fusing for Text-to-SQL. arXiv:2402.10663. Retrieved from <https://arxiv.org/abs/2402.10663>
- [109] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [110] Lijie Wang, Ao Zhang, Kun Wu, Ke Sun, Zhenghua Li, Hua Wu, Min Zhang, and Haifeng Wang. 2020. DuSQL: A large-scale and pragmatic Chinese text-to-SQL dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP'20)*. 6923–6935.
- [111] Tianshu Wang, Hongyu Lin, Xianpei Han, Le Sun, Xiaoyang Chen, Hao Wang, and Zhenyu Zeng. 2023. DBCopilot: Scaling natural language querying to massive databases. arXiv:2312.03463. Retrieved from <https://arxiv.org/abs/2312.03463>
- [112] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. arXiv:2203.11171. Retrieved from <https://arxiv.org/abs/2203.11171>
- [113] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. 2021. Finetuned language models are zero-shot learners. arXiv:2109.01652. Retrieved from <https://arxiv.org/abs/2109.01652>
- [114] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. arXiv:2206.07682. Retrieved from <https://arxiv.org/abs/2206.07682>
- [115] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [116] Lixia Wu, Peng Li, Junhong Lou, and Lei Fu. 2024. DataGpt-SQL-7B: An Open-Source Language Model for Text-to-SQL. arXiv:2409.15985. Retrieved from <https://arxiv.org/abs/2409.15985>
- [117] Hanchen Xia, Feng Jiang, Naihao Deng, Cunxiang Wang, Guojiang Zhao, Rada Mihalcea, and Yue Zhang. 2024. R³: “This is My SQL, Are You With Me?” A Consensus-Based Multi-Agent System for Text-to-SQL Tasks. arXiv:2402.14851. Retrieved from <https://arxiv.org/abs/2402.14851>
- [118] Wenxuan Xie, Gaochen Wu, and Bowen Zhou. 2024. Mag-sql: Multi-agent generative approach with soft schema linking and iterative sub-sql refinement for text-to-sql. arXiv:2408.07930. Retrieved from <https://arxiv.org/abs/2408.07930>
- [119] Xiangjin Xie, Guangwei Xu, Lingyan Zhao, and Ruijie Guo. 2025. OpenSearch-SQL: Enhancing Text-to-SQL with dynamic few-shot and consistency alignment. arXiv:2502.14913. Retrieved from <https://arxiv.org/abs/2502.14913>
- [120] Yuanzhen Xie, Xinzhou Jin, Tao Xie, MingXiong Lin, Liang Chen, Chenyun Yu, Lei Cheng, ChengXiang Zhuo, Bo Hu, and Zang Li. 2024. Decomposition for Enhancing Attention: Improving LLM-based Text-to-SQL through Workflow Paradigm. arXiv:2402.10671. Retrieved from <https://arxiv.org/abs/2402.10671>
- [121] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. arXiv:2304.12244. Retrieved from <https://arxiv.org/abs/2304.12244>
- [122] Xiaojun Xu, Chang Liu, and Dawn Song. 2017. Sqlnet: Generating structured queries from natural language without reinforcement learning. arXiv:1711.04436. Retrieved from <https://arxiv.org/abs/1711.04436>

- [123] Siqiao Xue, Caigao Jiang, Wenhui Shi, Fangyin Cheng, Keting Chen, Hongjun Yang, Zhiping Zhang, Jianshan He, Hongyang Zhang, Ganglin Wei, et al. 2023. Db-gpt: Empowering database interactions with private large language models. arXiv:2312.17449. Retrieved from <https://arxiv.org/abs/2312.17449>
- [124] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2025. Qwen2.5 Technical Report. arXiv:2412.15115. Retrieved from <https://arxiv.org/abs/2412.15115>
- [125] Sun Yang, Qiong Su, Zhishuai Li, Ziyue Li, Hangyu Mao, Chenxi Liu, and Rui Zhao. 2024. SQL-to-schema enhances schema linking in Text-to-SQL. In *Proceedings of the DEXA (1) (Lecture Notes in Computer Science, Vol. 14910)*. Springer, 139–145.
- [126] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems* 36 (2024).
- [127] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. arXiv:2210.03629. Retrieved from <https://arxiv.org/abs/2210.03629>
- [128] Tao Yu, Rui Zhang, He Yang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, et al. 2019. Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases. arXiv:1909.05378. Retrieved from <https://arxiv.org/abs/1909.05378>
- [129] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. arXiv:1809.08887. Retrieved from <https://arxiv.org/abs/1809.08887>
- [130] Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, et al. 2019. Sparc: Cross-domain semantic parsing in context. arXiv:1906.02285. Retrieved from <https://arxiv.org/abs/1906.02285>
- [131] Hongwei Yuan, Xiu Tang, Ke Chen, Lidan Shou, Gang Chen, and Huan Li. 2025. CogSQL: A cognitive framework for enhancing large language models in Text-to-SQL translation. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 25778–25786.
- [132] John M. Zelle and Raymond J. Mooney. 1996. Learning to parse database queries using inductive logic programming. In *Proceedings of the National Conference on Artificial Intelligence*. 1050–1055.
- [133] Bin Zhang, Yuxiao Ye, Guoqing Du, Xiaoru Hu, Zhishuai Li, Sun Yang, Chi Harold Liu, Rui Zhao, Ziyue Li, and Hangyu Mao. 2024. Benchmarking the Text-to-SQL capability of large language models: A comprehensive evaluation. arXiv:2403.02951. Retrieved from <https://arxiv.org/abs/2403.02951>
- [134] Chao Zhang, Yuren Mao, Yijiang Fan, Yu Mi, Yunjun Gao, Lu Chen, Dongfang Lou, and Jinshu Lin. 2024. FinSQL: Model-Agnostic LLMs-based Text-to-SQL Framework for Financial Analysis. arXiv:2401.10506. Retrieved from <https://arxiv.org/abs/2401.10506>
- [135] Hanchong Zhang, Ruisheng Cao, Lu Chen, Hongshen Xu, and Kai Yu. 2023. Act-sql: In-context learning for text-to-sql with automatically-generated chain-of-thought. arXiv:2310.17342. Retrieved from <https://arxiv.org/abs/2310.17342>
- [136] Hanchong Zhang, Ruisheng Cao, Hongshen Xu, Lu Chen, and Kai Yu. 2024. CoE-SQL: In-Context Learning for Multi-Turn Text-to-SQL with Chain-of-Edits. arXiv:2405.02712. Retrieved from <https://arxiv.org/abs/2405.02712>
- [137] Qinggang Zhang, Junnan Dong, Hao Chen, Wentao Li, Feiran Huang, and Xiao Huang. 2024. Structure Guided Large Language Model for SQL Generation. arXiv:2402.13284. Retrieved from <https://arxiv.org/abs/2402.13284>
- [138] Tingkai Zhang, Chaoyu Chen, Cong Liao, Jun Wang, Xudong Zhao, Hang Yu, Jianchao Wang, Jianguo Li, and Wenhui Shi. 2024. SQLfuse: Enhancing Text-to-SQL Performance through Comprehensive LLM Synergy. arXiv:2407.14568. Retrieved from <https://arxiv.org/abs/2407.14568>
- [139] Weixu Zhang, Yifei Wang, Yuanfeng Song, Victor Junqiu Wei, Yuxing Tian, Yiyan Qi, Jonathan H Chan, Raymond Chi-Wing Wong, and Haiqin Yang. 2024. Natural language interfaces for tabular data querying and visualization: A survey. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [140] Yi Zhang, Jan Deriu, George Katsogiannis-Meimarakis, Catherine Kosten, Georgia Koutrika, and Kurt Stockinger. 2023. ScienceBenchmark: A Complex Real-World Benchmark for Evaluating Natural Language to SQL Systems. arXiv:2306.04743. Retrieved from <https://arxiv.org/abs/2306.04743>
- [141] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. arXiv:2303.18223. Retrieved from <https://arxiv.org/abs/2303.18223>
- [142] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. arXiv:2306.05685. Retrieved from <https://arxiv.org/abs/2306.05685>
- [143] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic evaluation for text-to-SQL with distilled test suites. arXiv:2010.02840. Retrieved from <https://arxiv.org/abs/2010.02840>

- [144] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. arXiv:1709.00103. Retrieved from <https://arxiv.org/abs/1709.00103>
- [145] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. 2022. Least-to-most prompting enables complex reasoning in large language models. arXiv:2205.10625. Retrieved from <https://arxiv.org/abs/2205.10625>
- [146] Fan Zhou, Siqiao Xue, Danrui Qi, Wenhui Shi, Wang Zhao, Ganglin Wei, Hongyang Zhang, Caigai Jiang, Gangwei Jiang, Zhixuan Chu, and Faqiang Chen. 2024. DB-GPT-Hub: Towards open benchmarking Text-to-SQL empowered by large language models. arXiv:2406.11434. Retrieved from <https://arxiv.org/abs/2406.11434>
- [147] Xiaohu Zhu, Qian Li, Lizhen Cui, and Yongkang Liu. 2024. Large language model enhanced text-to-sql generation: A survey. arXiv:2410.06011. Retrieved from <https://arxiv.org/abs/2410.06011>

Received 12 November 2024; revised 5 May 2025; accepted 18 May 2025